

REAL-TIME DATA ANALYTICS • CAPSTONE PROJECT

LoyalSense Customer Loyalty Analytics



A complete, runnable project: profile a customer base with RFM, discover its natural segments with unsupervised clustering, score churn risk and lifetime value, then watch a live event stream move real customers between segments — and fire the right loyalty action the moment they move.

DELIVERABLE	Student handbook + full source code + dataset
LANGUAGE / STACK	Python 3.9+ pandas scikit-learn matplotlib joblib
DATASET	20 customer profiles + 16 live events (CSV)
KEY TECHNIQUE	Unsupervised K-Means + stateful stream processing
RUN COMMAND	<code>pip install -r requirements.txt → cd src → python main.py</code>
LEVEL	Beginner to Intermediate 8-12 hours

Everything needed to build and demonstrate this project is inside this zip file. No internet connection, database or API key required.

Table of Contents

1. Project Overview

- Problem statement
- Business context
- What makes this project real-time

2. Objectives and Learning Outcomes

3. Project Scope

4. Requirement Specification

- Functional requirements
- Non-functional requirements
- Software and hardware requirements

5. System Architecture

6. Dataset Description

- Customer data dictionary
- Event data dictionary
- Derived measures

7. Project Folder Structure

8. Methodology: The Five Stages

9. The Scoring and Action Rulebook

10. Code Walkthrough

- config.py
- data_loader.py
- rfm_engine.py
- eda.py
- model.py
- stream_simulator.py
- main.py

11. How to Run the Project

12. Expected Output

13. Results and Interpretation

14. Testing Checklist

15. Common Errors and Fixes

16. Viva / Interview Questions with Answers

17. Extension Ideas

18. Evaluation Rubric

19. Glossary

1 Project Overview

Build a system that knows which customers matter, which ones are quietly leaving, and what to do about each of them — and that updates its answer the instant a customer does anything at all.

1.1 Problem statement

A retail business has thousands of customers and treats them all the same. Everyone gets the same email, the same discount, the same nothing. Three problems follow:

- **The best customers are not recognised.** The person who has spent five lakh rupees over four years receives the same generic voucher as somebody who bought once and never returned.
- **Departures are only noticed in hindsight.** A customer who used to buy monthly and has not bought for six months has almost certainly gone, and nobody flagged it while there was still time.
- **Retention spend is untargeted.** Discounts go to people who would have bought anyway, while genuinely at-risk high-value customers get nothing.

This project builds the analytics layer that fixes all three: it profiles every customer, groups them, scores their risk, values them in rupees, and reacts in real time.

1.2 Business context

Question the marketing team asks	How the project answers it
Who are my best customers?	RFM scoring turns raw history into three 1–5 scores, and a rule grid turns those into named segments such as Champions and At Risk.
What natural groups exist that I have not thought of?	K-Means clustering finds structure in the data with no labels and no preconceptions, then each cluster is profiled and named.
Who is about to leave?	An explainable churn risk score from 0 to 100, built from recency, return rate, ratings and support tickets, with a written reason for every penalty.
Is this customer worth saving?	A projected customer lifetime value in rupees of gross margin, which shrinks as churn risk rises — so risk and value are compared on one screen.
What exactly should I do, right now?	An action matrix maps every segment to a concrete play and offer, with escalation to a human only when the projected margin justifies the cost.
Did anything just change?	Every incoming event re-scores the customer and reports any movement between segments, risk bands or clusters.

1.3 What makes this project real-time — and stateful

The two earlier projects in this series scored each arriving record on its own and then forgot it. This one cannot. An event such as 'C1005 bought something for Rs.15,800' means nothing in isolation — its significance depends entirely on what C1005 did before. The system therefore has to hold **keyed state**: one live profile per customer, updated in place.

Real loyalty platform	This project	Where in the code
Events arrive from the till, app and website over Kafka	<code>live_events.csv</code> is read row by row with a <code>time.sleep()</code> gap	<code>stream_simulator.run()</code>
A state store holds the current profile of every customer	A dictionary of <code>CustomerState</code> objects keyed by <code>customer_id</code>	<code>CustomerState</code>

Real loyalty platform	This project	Where in the code
The event is applied to that customer's running aggregates	<code>state.apply(event)</code> mutates frequency, monetary, recency, returns, tickets or rating	<code>CustomerState.apply</code>
The profile is re-scored against frozen model artefacts	RFM quintile boundaries and the fitted K-Means are loaded once and reused for every event	<code>evaluate()</code>
The platform detects a change of state	Before and after snapshots are compared to spot segment, risk-band and cluster movement	<code>migration()</code>
A campaign is triggered	The action matrix returns a play, an offer and a point multiplier	<code>rfm_engine.decide_action</code>
Dashboards refresh live counters	<code>RollingKPI</code> recomputes in $O(1)$ after every event	<code>stream_simulator.py</code>

The capability that only state gives you

A stateless scorer can tell you a customer *is* a Champion. Only a stateful one can tell you they **just became** a Champion, or that they just stopped being one. Segment migration is the single most actionable signal in loyalty analytics, and it is impossible to compute without remembering what came before. Replacing `for _, row in events.iterrows()` with a Kafka consumer, and the dictionary with RocksDB, is the only change needed to make this production-grade.

2 Objectives and Learning Outcomes

2.1 Project objectives

#	Objective
O1	Ingest a customer master and an event stream, and validate both against business rules such as 'returns cannot exceed orders'.
O2	Derive the three RFM measures plus the supporting ratios that make segments interpretable.
O3	Convert raw measures into 1–5 quintile scores using boundaries learned once and then frozen.
O4	Map RFM scores to named business segments through an ordered, data-driven rule grid.
O5	Score churn risk on a 0–100 scale with a written reason for every penalty applied.
O6	Project customer lifetime value in rupees of gross margin, linked to churn risk.
O7	Discover the natural structure of the customer base with unsupervised K-Means, choosing k with the elbow curve and the silhouette score, then name each cluster in business language.
O8	Maintain keyed state for every customer and update it in place as events arrive.
O9	Detect and report segment, risk-band and cluster migrations, and trigger the matching loyalty action.
O10	Produce an auditable output set: charts, a saved model, a scored customer file, an action log and a summary report.

2.2 What you will learn

Area	Skills demonstrated
Customer analytics	RFM, quintile scoring, segment grids, churn risk, customer lifetime value, the Pareto check, loyalty point economics
Unsupervised learning	Working with no target column at all. K-Means, why scaling is mandatory for distance-based algorithms, the elbow curve, the silhouette score, and how to name a cluster once you have one
Evaluating the unevaluable	There is no accuracy score here. You will learn the three things that replace it: a numeric separation measure, business interpretability, and stability under re-runs
Stateful stream processing	Keyed state, in-place aggregate updates, before/after snapshots, change detection, and O(1) rolling KPIs — the model used by Kafka Streams and Flink
Rules as data	An ordered segment grid, banded penalty tables and an action matrix, all expressed as data structures in <code>config.py</code> rather than as branching code
pandas	<code>quantile</code> , <code>np.searchsorted</code> , <code>groupby.agg</code> with named aggregations, <code>apply(result_type="expand")</code> , <code>crosstab</code>
Visualisation	Ranked horizontal bars with annotations, a bubble scatter map with point labels, a risk-versus-value quadrant chart with threshold lines, and grouped bars
Critical analysis	Why a rule-based segment and a discovered cluster disagree, and why annualising a three-month customer inflates their lifetime value

3 Project Scope

In scope	Out of scope
CSV ingestion of a customer master and an event stream	Live connection to a CRM, POS or e-commerce platform
RFM scoring with frozen quintile boundaries	Weighted or decay-based RFM variants
Rule-based segment naming from an ordered grid	Regulatory or accounting certification of the segments
Unsupervised K-Means clustering with k chosen by silhouette	Hierarchical, DBSCAN or Gaussian-mixture clustering
Explainable churn risk scoring from behavioural signals	A supervised churn model — there are no churn labels in this dataset
Simple deterministic CLV projection	Probabilistic CLV models such as BG/NBD and Gamma-Gamma
Stateful real-time processing of an event stream	A production broker, a durable state store or exactly-once semantics
A loyalty action matrix with escalation economics	Campaign execution, email delivery or offer redemption tracking
An optional Streamlit dashboard	Authentication, role-based access or deployment

Deliberately small dataset

Twenty customers is far below what a real segmentation needs (tens of thousands, so that quintiles are stable and clusters are meaningful). It is intentional: you can read every row, verify every score by hand, and still exercise the complete pipeline. Section 13 states precisely which conclusions this supports and which it does not.

4 Requirement Specification

This is the contract the code must satisfy. Every requirement is implemented, and the module that implements it is named so you can verify it yourself.

4.1 Functional requirements

ID	Requirement	Implemented in
FR-01	The system shall load the customer master and the event stream from CSV and abort with a clear message if either is missing.	<code>data_loader.load_csv</code>
FR-02	The system shall detect duplicate customer IDs and keep only the first record for each.	<code>data_loader.validate</code>
FR-03	The system shall exclude customers with no orders rather than inventing a purchase history for them.	<code>data_loader.validate</code>
FR-04	The system shall enforce that returns never exceed orders.	<code>data_loader.validate</code>
FR-05	The system shall derive recency, frequency and monetary value against a fixed analysis date.	<code>data_loader.derive_rfm</code>
FR-06	The system shall derive tenure, average order value, return rate and orders per year, annualising over a minimum window of one year.	<code>data_loader.derive_rfm</code>
FR-07	The system shall learn quintile boundaries once from the baseline population and reuse them unchanged for all later scoring.	<code>rfm_engine.build_score_bins</code>
FR-08	The system shall convert recency, frequency and monetary value into 1-5 scores, with the recency scale inverted.	<code>rfm_engine.score_rfm</code>
FR-09	The system shall assign a named segment using an ordered rule grid where the first match wins.	<code>rfm_engine.segment_for</code>
FR-10	The system shall score churn risk from 0 to 100 and return a written reason for every penalty applied.	<code>rfm_engine.churn_risk</code>
FR-11	The system shall project customer lifetime value in rupees of gross margin, reducing the horizon as churn risk rises.	<code>rfm_engine.clv</code>
FR-12	The system shall select a loyalty action, an offer and a point multiplier for every customer, escalating to a human only when risk is HIGH and value clears a threshold.	<code>rfm_engine.decide_action</code>
FR-13	The system shall report segment revenue, churn bands, the Pareto concentration and correlations with spend.	<code>eda.print_summary</code>
FR-14	The system shall generate and save at least four analytical charts as PNG files.	<code>eda.build_charts</code>
FR-15	The system shall evaluate every candidate cluster count, reporting inertia and silhouette for each.	<code>model.choose_k</code>
FR-16	The system shall scale features before clustering, because K-Means measures straight-line distance.	<code>model.build_pipeline</code>
FR-17	The system shall profile each cluster and assign it a unique business name.	<code>model.profile_clusters</code>
FR-18	The system shall persist the fitted pipeline together with the cluster names and metrics.	<code>model.train</code>
FR-19	The system shall hold one live state object per customer and update it in place as events arrive.	<code>CustomerState</code>
FR-20	The system shall handle four event types: purchase, return, support ticket and review.	<code>CustomerState.apply</code>
FR-21		<code>stream_simulator.migration</code>

ID	Requirement	Implemented in
	The system shall compare before and after snapshots and report segment, risk-band and cluster migrations.	
FR-22	The system shall award loyalty points on purchases at the multiplier of the customer's current segment.	<code>CustomerState.award_points</code>
FR-23	The system shall maintain rolling KPIs updated after every event.	<code>RollingKPI</code>
FR-24	The system shall write every event's before state, after state, movement and action to an audit CSV.	<code>stream_simulator.run</code>
FR-25	The system shall write a plain-text summary report and a fully scored customer file.	<code>main.write_summary</code>
FR-26	The system shall expose command-line flags to skip the delay and to skip chart generation.	<code>main.main</code>

4.2 Non-functional requirements

ID	Category	Requirement
NFR-01	Explainability	No score may be produced without a reason, and no action without a segment that justifies it.
NFR-02	Configurability	Every threshold, rule and action lives in <code>config.py</code> . No magic numbers anywhere else.
NFR-03	Score stability	Quintile boundaries are frozen after training, so a customer's score can only change because of their own behaviour.
NFR-04	Modularity	Each module runs standalone via its <code>__main__</code> block.
NFR-05	Reproducibility	A fixed <code>random_state</code> on K-Means and a fixed analysis date make every run identical.
NFR-06	Performance	Per-event processing and each KPI update are O(1); the full run finishes in under 5 seconds with <code>--fast</code> .
NFR-07	Auditability	Every run leaves charts, a model, a scored customer file, an action log and a summary on disk.
NFR-08	Portability	Runs on Windows, macOS, Linux and Colab; matplotlib uses the headless Agg backend.
NFR-09	Robustness	Events for unknown customers are skipped with a message rather than crashing the stream.

4.3 Software and hardware requirements

Item	Requirement
Operating system	Windows 10+, macOS 11+, or any modern Linux
Python	3.9 or newer (3.10 / 3.11 recommended)
RAM	2 GB is more than sufficient
Disk	About 60 MB including the Python libraries
Internet	Needed once, only to <code>pip install</code> the libraries. The project itself runs fully offline.
Editor	VS Code, PyCharm, Jupyter or plain IDLE
Libraries	pandas, numpy, scikit-learn, matplotlib, joblib (streamlit only for the optional dashboard)

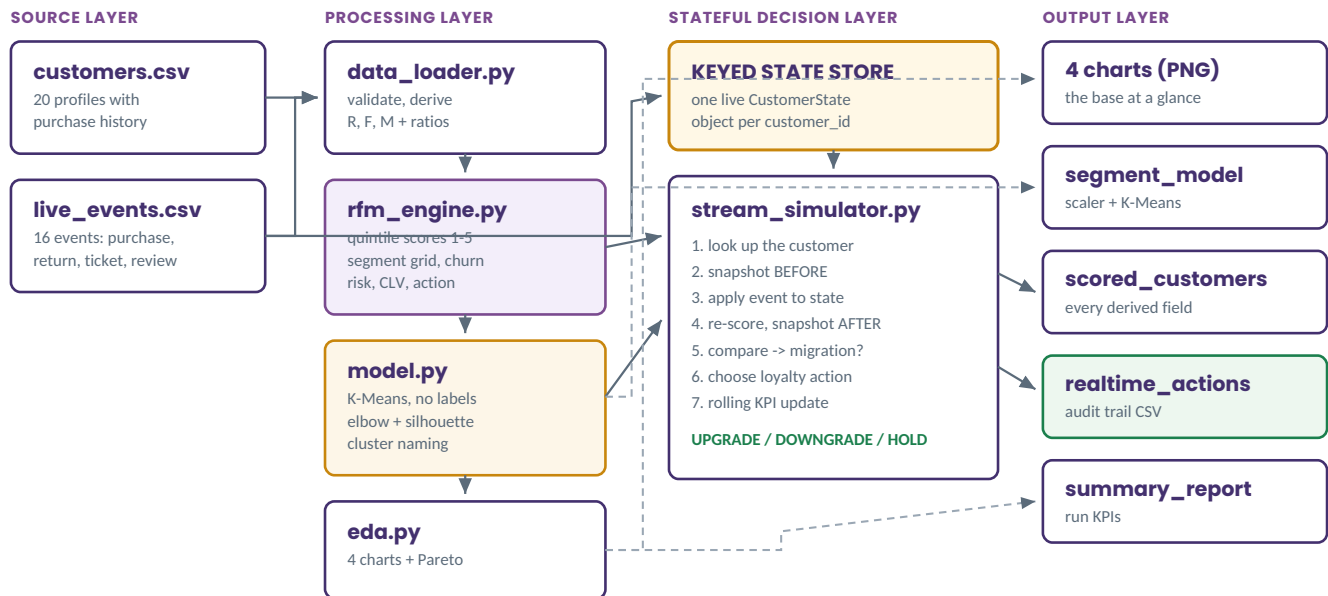
REQUIREMENTS.TXT

```
pandas>=2.0
numpy>=1.24
scikit-learn>=1.3
matplotlib>=3.7
```

```
joblib>=1.3  
# optional, only for app/dashboard.py  
# streamlit>=1.30
```

5 System Architecture

Four layers. The third one is different from anything in a stateless pipeline: it owns a **state store** that survives between events and is mutated by them.



Solid arrows = live data flow. Dashed arrows = artefacts written to disk.
The state store is what makes this different from a stateless pipeline: events mutate a profile that persists between events.

Layer	Responsibility	Modules
Source	Provide the customer master and the event feed	data/*.csv
Processing	Derive measures, score and segment, discover clusters, produce analytics	data_loader.py, rfm_engine.py, model.py, eda.py
Stateful decision	Hold one live profile per customer, apply events to it, detect movement, choose the action, track live KPIs	stream_simulator.py
Output	Persist everything an analyst or an auditor would ask for	outputs/

Two segmentations, deliberately

The rule grid and the clustering both divide the same customers, and they do not always agree. That is a feature, not a bug. The rule grid is *prescriptive* — it encodes what the business has decided matters, and its labels can go straight into a campaign brief. The clustering is *descriptive* — it reports the structure that is actually in the data, including groupings nobody wrote a rule for. Section 13.4 compares them directly and shows what the disagreements reveal.

6 Dataset Description

Two CSV files ship with the project. One describes who the customers are; the other describes what they are doing right now.

File	Rows	Role
data/customers.csv	20	The customer master: one row per customer with their accumulated purchase history. Used for scoring, EDA and clustering, and loaded into the state store when the stream starts.
data/live_events.csv	16	The live feed: one row per interaction on 1 April 2026, arriving in chronological order.

6.1 Customer data dictionary

Column	Type	Description
customer_id	string	Unique key, also the state-store key
name	string	Display name
city	category	Location, carried for reporting
signup_date	date	When the relationship began; drives tenure
last_purchase_date	date	Drives recency , the single most predictive loyalty signal
total_orders	int	Lifetime order count = frequency
total_spend	int	Lifetime spend in rupees = monetary
returns	int	Orders returned; drives the return rate
support_tickets	int	Complaints raised; a churn signal
avg_rating	float	Average review score out of 5
loyalty_points	int	Current points balance

DATA/CUSTOMERS.CSV (EXTRACT)

```
customer_id,name,city,signup_date,last_purchase_date,total_orders,total_spend,returns,support_tickets,avg_rating,loyalty_points
C1001,Aarav Sharma,Bengaluru,2023-02-14,2026-03-28,48,286000,2,1,4.7,5720
C1002,Diya Nair,Kochi,2022-11-05,2026-03-25,62,412000,3,2,4.8,8240
C1003,Rohan Gupta,Delhi,2024-06-20,2026-03-30,21,98000,1,0,4.5,1960
C1004,Ananya Iyer,Chennai,2023-08-11,2026-02-18,35,175000,2,1,4.4,3500
C1005,Vivaan Reddy,Hyderabad,2022-05-30,2025-11-12,54,321000,4,3,4.1,6420
C1006,Ishita Bose,Kolkata,2023-01-09,2025-09-05,29,143000,3,4,3.6,2860
C1007,Kabir Malhotra,Mumbai,2025-12-15,2026-03-29,6,31000,0,0,4.6,620
```

6.2 Event data dictionary

Column	Description
event_id	Unique event reference
timestamp	When the interaction happened
customer_id	Which state-store entry this event belongs to
event_type	One of PURCHASE, RETURN, SUPPORT_TICKET or REVIEW

Column	Description
amount	Rupees, for PURCHASE and RETURN only; 0 otherwise
rating	1 to 5, for REVIEW only. Deliberately sparse — it is left blank rather than filled with a fake value, which teaches you to handle optional columns honestly
channel	app, web, store, phone or email
detail	Free text carried through to the audit log

DATA/LIVE_EVENTS.CSV (EXTRACT)

```
event_id,timestamp,customer_id,event_type,amount,rating,channel,detail
E001,2026-04-01 09:12,C1012,PURCHASE,8400,,app,order #A8821
E002,2026-04-01 09:38,C1007,PURCHASE,6200,,web,order #A8822
E003,2026-04-01 10:05,C1005,PURCHASE,15800,,app,order #A8823
E004,2026-04-01 10:41,C1003,RETURN,4200,,web,return #R311
E005,2026-04-01 11:16,C1012,SUPPORT_TICKET,0,,phone,delivery delayed by 6 days
E006,2026-04-01 11:52,C1016,PURCHASE,9600,,app,order #A8824
E007,2026-04-01 12:20,C1014,RETURN,3100,,store,return #R312
```

6.3 Derived measures

Measure	Formula	Why it matters
recency_days	analysis date – last purchase date	How long since they last bought. Lower is better , which is why its score is inverted.
frequency	total orders	How often they buy. Correlates +0.988 with spend in this dataset.
monetary	total spend	How much they are worth historically.
tenure_days	analysis date – signup date	How long the relationship has run.
avg_order_value	monetary / frequency	Basket size. Separates 'buys often, buys small' from the reverse.
return_rate	returns / frequency	A dissatisfaction signal and a margin killer.
orders_per_year	frequency / max(tenure in years, 1.0)	Purchase rate, used for CLV. The minimum of one year is deliberate — see below.

The CLV inflation bug this avoids

Kabir Malhotra signed up 107 days ago and has placed 6 orders. Divide 6 by 0.29 years and you claim he buys **21 times a year** — more than any Champion in the base — and his projected lifetime value comes out at Rs.1,01,587, above Diya Nair who has actually spent Rs.4,12,000. Annualising over a minimum window of one full year gives 6 orders a year and a sane Rs.29,762. Extrapolating a short history is one of the most common and most expensive mistakes in customer analytics.

7 Project Folder Structure

FOLDER TREE

```

LoyalSense/
|
+-- README.md                quick start guide
+-- requirements.txt          library versions
+-- LoyalSense_Customer_Loyalty_Analytics.pdf  this handbook
|
+-- data/
|   +-- customers.csv        20 customers with their purchase history
|   +-- live_events.csv      16 events for the real-time feed
|
+-- src/
|   +-- config.py            paths, segment grid, churn rules, actions
|   +-- data_loader.py       ingestion, validation, RFM derivation
|   +-- rfm_engine.py        scores, segments, churn risk, CLV, actions
|   +-- eda.py               console statistics + 4 matplotlib charts
|   +-- model.py             K-Means, elbow + silhouette, cluster naming
|   +-- stream_simulator.py  stateful loop with migration detection
|   +-- main.py              orchestrator <-- RUN THIS
|
+-- app/
|   +-- dashboard.py         OPTIONAL Streamlit control room
|
+-- sample_outputs/          reference results, for comparison only
|   +-- console_output.txt   the full console log of a reference run
|   +-- ... the same 8 files listed below
|
+-- outputs/                  EMPTY until you run the project
    +-- 01_segment_revenue.png
    +-- 02_rfm_map.png
    +-- 03_churn_vs_value.png
    +-- 04_segment_profiles.png
    +-- segment_model.joblib
    +-- scored_customers.csv
    +-- realtime_actions.csv
    +-- summary_report.txt
  
```

Why src/ has no package prefix

The modules import each other plainly (`import config`) and you run `python main.py` from inside `src/` . This keeps imports readable for beginners. In a production repository you would add an `__init__.py` , turn `src` into a package and import as `from src import config` .

8 Methodology: The Five Stages

Running `python main.py` executes these five stages in order, each printed with a banner so you can follow along.

Stage	Name	Module	What happens
Stage 1	Ingestion and RFM derivation	<code>data_loader.py</code>	Read and validate the customer master, then derive recency, frequency, monetary value, tenure, average order value, return rate and the annualised purchase rate.
Stage 2	Scoring and segmentation	<code>rfm_engine.py</code>	Learn the quintile boundaries once, convert the three measures into 1-5 scores, map those to a named segment through the rule grid, score churn risk with written reasons, project CLV, and select a loyalty action and offer.
Stage 3	Exploratory analysis	<code>eda.py</code>	Report revenue and CLV per segment, the churn band split, the Pareto concentration and the correlations with spend; save four charts.
Stage 4	Unsupervised clustering	<code>model.py</code>	Scale the features, evaluate every candidate k on inertia and silhouette, fit K-Means at the best k, profile each cluster, name it, and persist the bundle.
Stage 5	Stateful real-time stream	<code>stream_simulator.py</code>	Build the state store, then for each arriving event: snapshot before, apply the event, re-score, snapshot after, detect migrations, award points, choose the action, update rolling KPIs and append to the audit log.

8.1 Data flow in one sentence per hop

1. CSV → `DataFrame` (11 columns) → validated (no duplicates, no zero-order customers, returns clipped).
2. Validated frame → enriched frame with the three RFM measures and four supporting ratios.
3. Enriched frame → quintile boundaries (learned once) → R, F and M scores → RFM code → named segment.
4. Scored frame → churn risk and reasons → CLV → loyalty action, offer and point multiplier.
5. Scored frame → scaled feature matrix → K-Means at the chosen k → cluster labels and names → `segment_model.joblib`.
6. Scored frame → one `CustomerState` per customer, held in a dictionary keyed by `customer_id`.
7. Event → look up state → snapshot → mutate → re-score → compare → migration, action, points → KPI update → log row.

9 The Scoring and Action Rulebook

Every boundary, rule and offer below lives in `config.py`. Change the loyalty programme by editing one file and re-running.

9.1 Quintile scoring

Each measure is cut into five equal-sized groups and scored 1 to 5. The boundaries below were learned from the 20 customers in the baseline and are then **frozen**.

Measure	Quintile cut-points	Direction	Score 5 means
recency_days	4.8, 10.4, 52.8, 172.0	Inverted — lower is better	Bought within about 5 days
frequency	15.6, 21.6, 30.6, 49.2	Higher is better	More than 49 lifetime orders
monetary	69,800; 94,000; 146,600; 293,000	Higher is better	More than Rs.2,93,000 spent

Why the boundaries must be frozen

If the quintiles were recomputed on every event, a customer's score could fall without them doing anything at all — purely because somebody else bought more and shifted the cut-points. Scores would stop being comparable over time and every segment migration would become meaningless. Learn once, then freeze. This is the same principle as fitting a scaler on training data only.

9.2 The segment rule grid

An ordered list. The first rule whose three ranges all match wins, so the most valuable and most urgent segments are tested first.

Order	Segment	R	F	M	Meaning
1	Champions	4-5	4-5	4-5	Recent, frequent, high spend. Your best customers.
2	Cannot Lose Them	1-2	4-5	4-5	Used to be Champions and have gone quiet. The most urgent segment in the whole grid.
3	Loyal Customers	3-5	3-5	3-5	Solid, dependable, worth growing.
4	At Risk	1-2	3-5	1-5	Bought often, now silent.
5	Potential Loyalist	4-5	2-3	1-5	Recent and building a habit.
6	New Customers	4-5	1	1-5	Just arrived; onboarding matters.
7	Promising	3-5	1-2	1-5	Early signs of engagement.
8	Needs Attention	3	3	1-5	Middling on everything and drifting.
9	Hibernating	2	1-2	1-5	Low value and slipping away.
10	Lost	1	1-2	1-5	Gone, and not economical to chase.
11	Others	1-5	1-5	1-5	Catch-all so no customer is ever unclassified.

Order is the whole design

'Cannot Lose Them' is tested second, before 'At Risk', because a lapsed big spender needs a phone call while a lapsed small spender needs an email. Move that rule down the list and your most valuable lapsed customers quietly get the cheap treatment. Reordering the grid changes the business, and nothing else in the code has to change.

9.3 Churn risk penalties

Signal	Bands	Penalty
Days since last purchase	≤ 30 / ≤ 60 / ≤ 120 / ≤ 240 / more	0 / +8 / +20 / +35 / +50
Return rate	$\leq 5\%$ / $\leq 10\%$ / $\leq 18\%$ / more	0 / +6 / +14 / +22
Average rating	≥ 4.5 / ≥ 4.0 / ≥ 3.5 / below	0 / +5 / +12 / +20
Support tickets	+4 each, capped at 20	0 to +20

Score	Band	Meaning
0-25	LOW	Healthy relationship
26-50	MEDIUM	Drifting; worth a cheap intervention
51-100	HIGH	Leaving or already gone

9.4 Customer lifetime value**RFM_ENGINE.CLV**

```
remaining_years = 3.0 x (1 - churn_score / 100)
CLV = avg_order_value x orders_per_year x remaining_years x 0.32
```

The margin rate of 32% turns revenue into profit, and the horizon is capped at three years because no retailer can credibly project further. Linking the horizon to churn risk is what makes the number actionable: a customer at 100 risk has a CLV of zero, which is the correct answer — there is nothing left to lose.

9.5 The action matrix

Segment	Action	Offer	Points
Champions	Invite to the VIP tier and early-access drops	Free express delivery for 12 months	3.0x
Cannot Lose Them	Personal win-back call from an account manager	25% voucher, 14 days	3.0x
Loyal Customers	Cross-sell from the adjacent category	Bonus points on the next 3 orders	2.0x
At Risk	Automated win-back email sequence	15% voucher, 21 days	2.0x
Potential Loyalist	Nudge towards a subscription plan	Free trial month	1.5x
New Customers	Onboarding journey and product education	10% off the second order	1.5x
Promising	Encourage a second category purchase	Bundle discount	1.5x
Needs Attention	Re-engagement offer before recency worsens	10% voucher, 30 days	1.0x
Hibernating	Low-cost reactivation email	Free delivery on any order	1.0x
Lost	Move to a quarterly newsletter only	No offer — not economical	1.0x

Escalation economics

On top of the segment play, a customer is escalated to a human only when churn risk is **HIGH** and projected CLV is at least Rs. 15,000 of margin. Both conditions matter. Escalating every high-risk customer would flood the retention desk with people worth nothing; escalating every valuable customer would waste calls on people who are perfectly happy. In the live run this fires exactly twice out of sixteen events.

9.6 A worked example: event E003

Vivaan Reddy (C1005) had spent Rs.3,21,000 over 54 orders but had not bought for 140 days. At 10:05 he places an order for Rs.15,800.

Step	Before the event	After the event
Recency	140 days	0 days
Frequency	54 orders	55 orders
Monetary	Rs. 3,21,000	Rs. 3,36,800
RFM scores	R2 F5 M5	R5 F5 M5
Segment	Cannot Lose Them	Champions
Churn risk	58 / 100 HIGH	23 / 100 LOW
Projected CLV	Rs. 33,720	Rs. 64,870 (+31,150)
Action	Personal win-back call	Invite to the VIP tier and early-access drops
Points	-	+948 at the 3.0x Champion multiplier

This is what stateful processing buys you

A stateless system would have looked at a Rs.15,800 order and logged a Rs.15,800 order. The stateful system reports something far more useful: *a customer we had written off as nearly lost has just come back, their risk has collapsed from HIGH to LOW, they are worth Rs.31,150 more than they were an hour ago, and they should be moved out of the win-back campaign and into the VIP tier immediately.* None of that is computable without remembering what came before.

10 Code Walkthrough

Seven modules, about 850 lines in total. Each is printed in full and then explained. Read them in this order — each builds on the one before it.

Order	Module	Lines	One-line job
1	config.py	~135	Every path, rule and offer in one place
2	data_loader.py	~110	Read, validate and derive the RFM measures
3	rfm_engine.py	~180	Scores, segments, churn risk, CLV and actions
4	eda.py	~150	Describe the base in numbers and four charts
5	model.py	~150	Unsupervised clustering with no labels at all
6	stream_simulator.py	~230	Keyed state and the real-time event loop
7	main.py	~120	Wire the five stages together

10.1 config.py — the single source of truth

A marketing manager will want to argue about the segment boundaries, the offers and the escalation threshold. None of those arguments should require reading analytics code. **If it is a policy somebody could dispute, it lives here.**

SRC/CONFIG.PY

```
"""
config.py
-----
One place for every path, threshold, segment rule and loyalty action used by
the project. If it is a number or a policy a marketing manager could argue
about, it lives here and nowhere else.
"""

from datetime import date
from pathlib import Path

# -----
# 1. FOLDER LAYOUT
# -----
BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "outputs"

CUSTOMERS_CSV = DATA_DIR / "customers.csv"
EVENTS_CSV = DATA_DIR / "live_events.csv"

MODEL_FILE = OUTPUT_DIR / "segment_model.joblib"
SCORED_CUSTOMERS = OUTPUT_DIR / "scored_customers.csv"
EVENT_LOG = OUTPUT_DIR / "realtime_actions.csv"
SUMMARY_REPORT = OUTPUT_DIR / "summary_report.txt"

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# -----
# 2. ANALYSIS DATE
# -----
# Recency is measured against this date. Hard-coding it keeps the whole project
# reproducible; a live system would use date.today().
AS_OF_DATE = date(2026, 4, 1)
```

```

# -----
# 3. RFM SCORING
# -----
RFM_BINS = 5          # quintiles -> scores 1 to 5

# Features handed to the clustering model. Note there is NO target column
# anywhere in this project: segmentation is unsupervised.
CLUSTER_FEATURES = [
    "recency_days",
    "frequency",
    "monetary",
    "avg_order_value",
    "return_rate",
    "tenure_days",
]

K_RANGE = range(2, 7)    # candidate cluster counts to evaluate
RANDOM_STATE = 42

# -----
# 4. SEGMENT RULE GRID
# -----
# Ordered list; the first rule whose ranges all match wins.
# (segment name, r_min, r_max, f_min, f_max, m_min, m_max)
SEGMENT_RULES = [
    ("Champions",          4, 5, 4, 5, 4, 5),
    ("Cannot Lose Them",  1, 2, 4, 5, 4, 5),
    ("Loyal Customers",   3, 5, 3, 5, 3, 5),
    ("At Risk",           1, 2, 3, 5, 1, 5),
    ("Potential Loyalist", 4, 5, 2, 3, 1, 5),
    ("New Customers",     4, 5, 1, 1, 1, 5),
    ("Promising",         3, 5, 1, 2, 1, 5),
    ("Needs Attention",   3, 3, 3, 3, 1, 5),
    ("Hibernating",       2, 2, 1, 2, 1, 5),
    ("Lost",              1, 1, 1, 2, 1, 5),
    ("Others",            1, 5, 1, 5, 1, 5), # catch-all, must stay last
]

# -----
# 5. CHURN RISK RULES (additive penalties, capped at 100)
# -----
REGENCY_BANDS = [      # (max days since last purchase, penalty)
    (30, 0),
    (60, 8),
    (120, 20),
    (240, 35),
    (10_000, 50),
]

RETURN_RATE_BANDS = [  # (max return rate, penalty)
    (0.05, 0),
    (0.10, 6),
    (0.18, 14),
    (1.00, 22),
]

RATING_BANDS = [       # (minimum average rating, penalty)
    (4.5, 0),
    (4.0, 5),
    (3.5, 12),
    (0.0, 20),
]

# How many past ratings a new review is averaged against. Order count is a
# poor proxy: a customer with 74 orders has not left 74 reviews, and using that
# weight would make a fresh 1-star review mathematically invisible.
RATING_MEMORY = 5

```

```

TICKET_PENALTY = 4          # points per open support ticket
TICKET_PENALTY_CAP = 20

CHURN_BANDS = [            # (max score, label)
    (25, "LOW"),
    (50, "MEDIUM"),
    (100, "HIGH"),
]

# -----
# 6. CUSTOMER LIFETIME VALUE
# -----
GROSS_MARGIN = 0.32        # share of revenue that is actually profit
MAX_HORIZON_YEARS = 3.0    # how far ahead the business is willing to project

# -----
# 7. LOYALTY ACTION MATRIX
# -----
# segment -> (action, offer, loyalty-point multiplier on a purchase)
ACTION_MATRIX = {
    "Champions": ("Invite to the VIP tier and early-access drops",
                  "Free express delivery for 12 months", 3.0),
    "Cannot Lose Them": ("Personal win-back call from an account manager",
                          "25% voucher, valid 14 days", 3.0),
    "Loyal Customers": ("Cross-sell from the adjacent category",
                        "Bonus points on the next 3 orders", 2.0),
    "At Risk": ("Automated win-back email sequence",
                "15% voucher, valid 21 days", 2.0),
    "Potential Loyalist": ("Nudge towards a subscription plan",
                           "Free trial month", 1.5),
    "New Customers": ("Onboarding journey and product education",
                      "10% off the second order", 1.5),
    "Promising": ("Encourage a second category purchase",
                  "Bundle discount", 1.5),
    "Needs Attention": ("Re-engagement offer before recency worsens",
                        "10% voucher, valid 30 days", 1.0),
    "Hibernating": ("Low-cost reactivation email",
                    "Free delivery on any order", 1.0),
    "Lost": ("Move to a quarterly newsletter only",
             "No offer - not economical", 1.0),
    "Others": ("Review manually", "No offer", 1.0),
}

# How "good" each segment is. Used only to decide whether a customer moving
# between segments has been upgraded or downgraded.
SEGMENT_RANK = {
    "Champions": 10,
    "Loyal Customers": 9,
    "Cannot Lose Them": 8,
    "Potential Loyalist": 7,
    "Promising": 6,
    "New Customers": 5,
    "Needs Attention": 4,
    "At Risk": 3,
    "Hibernating": 2,
    "Lost": 1,
    "Others": 0,
}

# Escalations applied on top of the segment action
# A personal call from a human costs money, so escalate only when the projected
# margin justifies it. CLV here is gross margin over at most three years.
HIGH_VALUE_CLV = 15_000    # rupees of projected margin
ESCALATE_ACTION = "Escalate to the retention desk within 24 hours"

BASE_POINTS_PER_RUPEE = 0.02 # 2 points per rupee spent, before multipliers

```

```
# -----
# 8. STREAM SETTINGS
# -----
STREAM_DELAY_SECONDS = 1.0
```

Construct	What it does and why it matters
AS_OF_DATE	Recency is measured against this fixed date rather than <code>date.today()</code> . Hard-coding it is what makes every number in this handbook reproducible; a live system would swap in today's date and nothing else would change.
SEGMENT_RULES	The whole segmentation policy expressed as an ordered list of tuples rather than an <code>if/elif</code> chain. Rules as data: you can reorder the business without touching logic, and the grid can be printed straight into a document.
RECENCY_BANDS and friends	Banded penalty tables, again as data. The same <code>_band_penalty</code> helper walks all of them.
ACTION_MATRIX	A dictionary from segment to (action, offer, point multiplier). Adding a campaign is a one-line change with no code path to test.
SEGMENT_RANK	Exists for one purpose only: deciding whether a move between two segments was an upgrade or a downgrade. Keeping it separate from <code>SEGMENT_RULES</code> means the ordering of the grid (which is about priority) never gets confused with the ranking (which is about quality).
CLUSTER_FEATURES	Exactly what the clustering may see. Note there is no target column anywhere in this file — that absence is the defining characteristic of the project.
RATING_MEMORY	How many past ratings a new review is averaged against. Explained in 10.6; it exists because using order count as the weight makes a fresh 1-star review mathematically invisible.

10.2 data_loader.py — ingestion and RFM derivation

Three jobs: get the data in, prove it is usable, and turn raw history into the measures everything else depends on.

SRC/DATA_LOADER.PY

```
"""
data_loader.py
-----
Stage 1: read the customer master and the event stream, prove they are usable,
and derive the raw RFM measurements that everything downstream depends on.

Note what is NOT here: there is no target column, no "churned = yes/no" label.
This project is unsupervised, so the data preparation has to be right - there
is no ground truth later to rescue a mistake.
"""

import pandas as pd

import config

def load_csv(path, **kwargs):
    """Read a CSV and fail loudly if it is missing."""
    if not path.exists():
        raise FileNotFoundError(f"Required data file not found: {path}")
    df = pd.read_csv(path, **kwargs)
    print(f"[data_loader] Loaded {len(df)} rows x {len(df.columns)} columns "
          f"from {path.name}")
    return df

def validate(df):
```

```

"""Data-quality gate for the customer master."""
report = {
    "customers": len(df),
    "duplicate_ids": int(df["customer_id"].duplicated().sum()),
    "missing_values": int(df.isna().sum().sum()),
    "zero_order_rows": int((df["total_orders"] <= 0).sum()),
    "returns_exceed_orders": int((df["returns"] > df["total_orders"]).sum()),
}

if report["duplicate_ids"]:
    df = df.drop_duplicates(subset="customer_id", keep="first")

# A customer with no orders has no RFM profile at all, so they are excluded
# rather than imputed. Inventing a purchase history would be worse.
if report["zero_order_rows"]:
    df = df[df["total_orders"] > 0]

# Returns can never exceed orders; clip rather than discard the customer.
df["returns"] = df[["returns", "total_orders"]].min(axis=1)

for col in ["avg_rating"]:
    if df[col].isna().any():
        df[col] = df[col].fillna(df[col].median())

print(f"[data_loader] Validation -> {report}")
return df, report

def derive_rfm(df):
    """
    Turn raw history into the three numbers every loyalty programme runs on.

    Recency - how many days since the customer last bought
    Frequency - how many orders they have placed in total
    Monetary - how much they have spent in total

    Plus the supporting ratios that make the clusters interpretable.
    """
    df = df.copy()
    as_of = pd.Timestamp(config.AS_OF_DATE)

    df["signup_date"] = pd.to_datetime(df["signup_date"])
    df["last_purchase_date"] = pd.to_datetime(df["last_purchase_date"])

    # The three pillars
    df["recency_days"] = (as_of - df["last_purchase_date"]).dt.days
    df["frequency"] = df["total_orders"]
    df["monetary"] = df["total_spend"]

    # Supporting measures
    df["tenure_days"] = (as_of - df["signup_date"]).dt.days
    df["avg_order_value"] = (df["monetary"] / df["frequency"]).round(0)
    df["return_rate"] = (df["returns"] / df["frequency"]).round(3)
    # Annualise over at least one full year. Dividing a 3-month-old customer's
    # 6 orders by 0.29 years would claim 21 orders a year and inflate their
    # lifetime value above a genuine Champion's - the classic CLV bug.
    years = (df["tenure_days"] / 365.25).clip(lower=1.0)
    df["orders_per_year"] = (df["frequency"] / years).round(2)

    print(f"[data_loader] Derived RFM for {len(df)} customers "
          f"(recency {df['recency_days'].min()}-{df['recency_days'].max()} days, "
          f"spend Rs.{df['monetary'].min():,} - Rs.{df['monetary'].max():,})")
    return df

def get_customers():
    """Full pipeline for the customer master."""

```

```

df, _ = validate(load_csv(config.CUSTOMERS_CSV))
return derive_rfm(df)

def get_events():
    """
    Load the live event stream, sorted chronologically.

    `rating` is deliberately sparse - only REVIEW events carry one - so it is
    read as a nullable float rather than being force-filled with a fake value.
    """
    df = load_csv(config.EVENTS_CSV, parse_dates=["timestamp"])
    df = df.sort_values("timestamp").reset_index(drop=True)
    df["amount"] = df["amount"].fillna(0).astype(float)
    counts = df["event_type"].value_counts().to_dict()
    print(f"[data_loader] Event mix -> {counts}")
    return df

if __name__ == "__main__":
    customers = get_customers()
    print(customers[["customer_id", "recency_days", "frequency", "monetary",
                    "avg_order_value", "return_rate"]].head(8).to_string(index=False))

    get_events()

```

Element	Explanation
Zero-order customers are dropped	A customer with no orders has no recency, no frequency and no average order value. Imputing a purchase history for them would invent behaviour that never happened, so they are excluded and the exclusion is reported. Choosing not to impute is a decision, and it should be a visible one.
<code>df[["returns", "total_orders"]].min(axis=1)</code>	Clips returns to the order count. A business rule violation is repaired rather than used as a reason to discard an otherwise good customer record.
Recency from a fixed date	<code>(as_of - last_purchase_date).dt.days</code> . Everything about loyalty analytics rests on this one number.
<code>.clip(lower=1.0)</code> on tenure years	The minimum one-year annualisation window from Section 6.3. Without it, a three-month-old customer's six orders become twenty-one a year and their CLV overtakes a genuine Champion's.
Sparse <code>rating</code> in the event file	Only REVIEW events carry a rating, so the column is left mostly empty rather than force-filled. <code>amount</code> , which every event type has a sensible zero for, is filled.
Events sorted by timestamp	The stream must be replayed in the order it happened, or every before/after comparison downstream becomes nonsense.

10.3 rfm_engine.py — the explainable half

Scores, segments, risk, value and actions. No machine learning at all — a manager could reproduce every number in a spreadsheet, which is exactly why it sits alongside the clustering rather than inside it.

SRC/RFM_ENGINE.PY

```

"""
rfm_engine.py
-----
Stage 2: the explainable half of the project.

Turns raw RFM numbers into scores, scores into a named segment, behaviour into
a churn risk, and a segment into a concrete loyalty action. No machine learning
anywhere in this file - a marketing manager could reproduce every number with a
spreadsheet, which is exactly why it sits alongside the clustering model rather
than inside it.
"""

```

```

import numpy as np
import pandas as pd

import config

# ----- RFM scoring
def build_score_bins(df):
    """
    Learn the quintile boundaries once, from the baseline population.

    This is the single most important idea in the module. If the boundaries
    were recomputed for every incoming event, a customer's score could change
    without the customer doing anything, purely because their peers moved.
    Freezing the cut-points makes scores comparable over time.
    """
    bins = {}
    for col in ["recency_days", "frequency", "monetary"]:
        # 5 quintiles -> 6 edges. Interior edges only; the ends are open.
        edges = df[col].quantile([0.2, 0.4, 0.6, 0.8]).tolist()
        bins[col] = [round(float(e), 2) for e in edges]
    print(f"[rfm_engine] Score boundaries learned from {len(df)} customers")
    for col, edges in bins.items():
        print(f"    {col:<14} quintile cut-points: {edges}")
    return bins

def _score(value, edges, reverse=False):
    """
    Map a value to 1-5 using pre-learned edges.

    reverse=True is used for recency, where a LOW number is GOOD, so the
    scale is flipped: 3 days since purchase should score 5, not 1.
    """
    rank = int(np.searchsorted(edges, value, side="right")) + 1 # 1..5
    return 6 - rank if reverse else rank

def score_rfm(row, bins):
    """Return the three 1-5 scores plus the concatenated RFM code."""
    r = _score(row["recency_days"], bins["recency_days"], reverse=True)
    f = _score(row["frequency"], bins["frequency"])
    m = _score(row["monetary"], bins["monetary"])
    return {"r_score": r, "f_score": f, "m_score": m, "rfm_code": f"{r}{f}{m}"}

# ----- segment mapping
def segment_for(r, f, m):
    """Walk the ordered rule grid and return the first segment that matches."""
    for name, r_lo, r_hi, f_lo, f_hi, m_lo, m_hi in config.SEGMENT_RULES:
        if r_lo <= r <= r_hi and f_lo <= f <= f_hi and m_lo <= m <= m_hi:
            return name
    return "Others"

# ----- churn risk
def _band_penalty(value, bands, higher_is_better=False):
    """Walk a (threshold, penalty) list and return the first match."""
    for threshold, penalty in bands:
        if higher_is_better and value >= threshold:
            return penalty
        if not higher_is_better and value <= threshold:
            return penalty
    return bands[-1][1]

```

```

def churn_risk(row):
    """
    Additive, fully explainable churn score from 0 (safe) to 100 (leaving).

    Returns the score, its band, and a written reason for every penalty so the
    number can always be defended to the customer or to a manager.
    """
    reasons, score = [], 0

    p = _band_penalty(row["recency_days"], config.RECENCY_BANDS)
    if p:
        reasons.append(f"{int(row['recency_days'])} days since last purchase ({+p})")
        score += p

    p = _band_penalty(row["return_rate"], config.RETURN_RATE_BANDS)
    if p:
        reasons.append(f"return rate {row['return_rate']:.0%} ({+p})")
        score += p

    p = _band_penalty(row["avg_rating"], config.RATING_BANDS, higher_is_better=True)
    if p:
        reasons.append(f"average rating {row['avg_rating']:.1f} ({+p})")
        score += p

    tickets = int(row["support_tickets"])
    if tickets:
        p = min(tickets * config.TICKET_PENALTY, config.TICKET_PENALTY_CAP)
        reasons.append(f"{tickets} support ticket(s) ({+p})")
        score += p

    score = min(score, 100)
    band = next(label for limit, label in config.CHURN_BANDS if score <= limit)

    return {
        "churn_score": score,
        "churn_band": band,
        "churn_reasons": "; ".join(reasons) if reasons else "no risk flags",
    }

# ----- CLV
def clv(row):
    """
    Projected gross margin over the remaining relationship.

    CLV = average order value
          x orders per year
          x expected remaining years
          x gross margin

    Expected remaining years shrinks as churn risk rises, which is what links
    the risk score to a rupee figure the business can act on.
    """
    remaining_years = config.MAX_HORIZON_YEARS * (1 - row["churn_score"] / 100)
    value = (row["avg_order_value"] * row["orders_per_year"]
             * remaining_years * config.GROSS_MARGIN)
    return {
        "expected_years": round(remaining_years, 2),
        "clv": round(float(value), 0),
    }

# ----- loyalty action
def decide_action(segment, churn_band, clv_value):
    """
    Pick the next best action.

```

The segment chooses the play; churn risk and value decide whether it is escalated to a human. A retention desk that receives every alert acts on none of them, so escalation is deliberately narrow.

"""

```
action, offer, multiplier = config.ACTION_MATRIX.get(
    segment, config.ACTION_MATRIX["Others"])
```

```
escalate = churn_band == "HIGH" and clv_value >= config.HIGH_VALUE_CLV
base_action = action
```

```
if escalate:
```

```
    action = f"{config.ESCALATE_ACTION}; then {base_action.lower()}"
```

```
return {
```

```
    "action": action,
```

```
    "base_action": base_action,
```

```
    "offer": offer,
```

```
    "point_multiplier": multiplier,
```

```
    "escalated": escalate,
```

```
}
```

```
# ----- frame-level API
```

```
def score_customers(df, bins=None):
```

```
    """Apply every rule above to a whole DataFrame and return the scored copy."""
```

```
    bins = bins or build_score_bins(df)
```

```
    rfm = df.apply(lambda r: score_rfm(r, bins), axis=1, result_type="expand")
```

```
    out = df.join(rfm)
```

```
    out["segment"] = out.apply(
```

```
        lambda r: segment_for(r["r_score"], r["f_score"], r["m_score"]), axis=1)
```

```
    risk = out.apply(churn_risk, axis=1, result_type="expand")
```

```
    out = out.join(risk)
```

```
    out = out.join(out.apply(clv, axis=1, result_type="expand"))
```

```
    plays = out.apply(
```

```
        lambda r: decide_action(r["segment"], r["churn_band"], r["clv"]),
```

```
        axis=1, result_type="expand")
```

```
    out = out.join(plays)
```

```
    counts = out["segment"].value_counts().to_dict()
```

```
    print(f"[rfm_engine] Scored {len(out)} customers -> {counts}")
```

```
    return out, bins
```

```
if __name__ == "__main__":
```

```
    import data_loader
```

```
    scored, _ = score_customers(data_loader.get_customers())
```

```
    print(scored[["customer_id", "rfm_code", "segment", "churn_score",
```

```
                  "churn_band", "clv"]].to_string(index=False))
```

Element	Explanation
<code>build_score_bins(df)</code>	The most important function in the module. It learns the four interior quintile edges for each measure, once, and returns them. Everything afterwards scores against these frozen numbers. Section 9.1 explains why recomputing them per event would destroy the meaning of a segment migration.
<code>np.searchsorted(edges, value, side="right")</code>	Finds which bucket a value falls into by binary search. Far cleaner than a chain of comparisons, and it works identically for one value or a whole column.
<code>reverse=True</code> for recency	6 - rank flips the scale. Three days since a purchase is excellent and must score 5, not 1. Forgetting this inversion is the single most common RFM bug.
<code>segment_for(r, f, m)</code>	

Element	Explanation
	Walks the ordered grid and returns on the first match. Eleven lines of code implement a policy that would otherwise be a fifty-line <code>if/elif</code> tree.
<code>churn_risk(row)</code>	Additive penalties, each appending a sentence to a <code>reasons</code> list as it is applied. Building the explanation <i>while</i> computing the score guarantees the two can never drift apart.
<code>clv(row)</code>	Ties the horizon to the risk score, so a customer at 100 risk is worth zero. That is the correct answer, not a bug: there is nothing left to retain.
<code>decide_action(...)</code>	Segment chooses the play; risk and value decide escalation. <code>base_action</code> is kept alongside the escalated string so the audit log can show both what was escalated and what the underlying play was.
<code>score_customers(df, bins=None)</code>	The optional <code>bins</code> argument is what lets the streaming layer reuse the exact boundaries learned at training time instead of silently re-learning them.

10.4 eda.py – understanding the base

Never cluster data you have not looked at. This module answers: who generates the revenue, how concentrated is it, and what actually correlates with spend?

SRC/EDA.PY

```
"""
eda.py
-----
Stage 3: describe the customer base in numbers, then in four pictures.

matplotlib only, with the Agg backend so it runs on a server, in Docker and in
Colab without trying to open a window.
"""

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

import config

SEGMENT_COLOUR = {
    "Champions": "#1b6b3a",
    "Loyal Customers": "#3f9d4f",
    "Potential Loyalist": "#7bbf6a",
    "Promising": "#a8d08d",
    "New Customers": "#4a90c2",
    "Needs Attention": "#e0a53a",
    "At Risk": "#e07b39",
    "Cannot Lose Them": "#c0392b",
    "Hibernating": "#9b7b8c",
    "Lost": "#7a7a7a",
    "Others": "#bdbdbd",
}

BAND_COLOUR = {"LOW": "#3f9d4f", "MEDIUM": "#e0a53a", "HIGH": "#c0392b"}

def print_summary(df):
    print("\n" + "=" * 72)
    print("EXPLORATORY DATA ANALYSIS")
    print("=" * 72)

    print(f"Customers           : {len(df)}")
    print(f"Total revenue       : Rs.{df['monetary'].sum():,}")
    print(f"Average order value : Rs.{df['avg_order_value'].mean():.0f}")
    print(f"Median recency      : {df['recency_days'].median():.0f} days")
```

```

print(f"Total projected CLV : Rs.{df['clv'].sum():.0f} of gross margin")

print("\n-- Segment breakdown --")
grp = (df.groupby("segment")
        .agg(customers=("customer_id", "size"),
              revenue=("monetary", "sum"),
              avg_recency=("recency_days", "mean"),
              avg_churn=("churn_score", "mean"),
              clv=("clv", "sum"))
        .sort_values("revenue", ascending=False))
total_rev = df["monetary"].sum()
for seg, r in grp.iterrows():
    print(f" {seg:<19} {int(r['customers']):2d} cust "
          f"Rs.{r['revenue']:>9.0f} ({r['revenue'] / total_rev:5.1%}) "
          f"recency {r['avg_recency']:5.0f}d churn {r['avg_churn']:4.0f} "
          f"CLV Rs.{r['clv']:>9.0f}")

print("\n-- Churn risk bands --")
for band in ["LOW", "MEDIUM", "HIGH"]:
    sub = df[df["churn_band"] == band]
    if len(sub):
        print(f" {band:<7} {len(sub):2d} customers "
              f"Rs.{sub['monetary'].sum():>9.0f} of historic revenue at stake")

print("\n-- The Pareto check --")
ranked = df.sort_values("monetary", ascending=False)
top20 = max(1, int(len(df) * 0.2))
share = ranked.head(top20)["monetary"].sum() / total_rev
print(f" Top {top20} customers ({top20 / len(df):.0%}) generate "
      f"{share:.1%} of all revenue")

print("\n-- Correlation with total spend --")
corr = df[["recency_days", "frequency", "avg_order_value", "tenure_days",
           "return_rate", "monetary"]].corr(numeric_only=True)["monetary"]
for name, v in corr.drop("monetary").sort_values(ascending=False).items():
    print(f" {name:<18} {v:+.3f}")

def _save(fig, name):
    path = config.OUTPUT_DIR / name
    fig.tight_layout()
    fig.savefig(path, dpi=130)
    plt.close(fig)
    print(f"[eda] saved {path.name}")

def build_charts(df):
    # 1. Customers and revenue per segment -----
    grp = (df.groupby("segment")
            .agg(customers=("customer_id", "size"), revenue=("monetary", "sum"))
            .sort_values("revenue"))
    fig, ax = plt.subplots(figsize=(7, 3.8))
    bars = ax.barh(grp.index, grp["revenue"] / 1000,
                   color=[SEGMENT_COLOUR.get(s, "#999") for s in grp.index])
    for bar, n in zip(bars, grp["customers"]):
        ax.text(bar.get_width() + 12, bar.get_y() + bar.get_height() / 2,
                f"{n} cust", va="center", fontsize=8)
    ax.set_xlabel("Historic revenue (thousand rupees)")
    ax.set_title("Revenue and customer count by RFM segment")
    ax.set_xlim(0, grp["revenue"].max() / 1000 * 1.22)
    _save(fig, "01_segment_revenue.png")

    # 2. The RFM map -----
    fig, ax = plt.subplots(figsize=(7, 4.2))
    for seg in df["segment"].unique():
        sub = df[df["segment"] == seg]
        ax.scatter(sub["recency_days"], sub["monetary"] / 1000,

```

```

        s=sub["frequency"] * 6 + 30,
        color=SEGMENT_COLOUR.get(seg, "#999"),
        alpha=0.85, edgecolors="white", label=seg)
for _, r in df.iterrows():
    ax.annotate(r["customer_id"][-3:], (r["recency_days"], r["monetary"] / 1000),
               fontsize=6, ha="center", va="center", color="white")
ax.set_xlabel("Recency (days since last purchase) - lower is better")
ax.set_ylabel("Monetary value (thousand rupees)")
ax.set_title("The RFM map (bubble size = purchase frequency)")
ax.legend(fontsize=7, ncol=2, loc="upper right")
_save(fig, "02_rfm_map.png")

```

```

# 3. Churn risk vs value -----
fig, ax = plt.subplots(figsize=(6.4, 4))
for band in ["LOW", "MEDIUM", "HIGH"]:
    sub = df[df["churn_band"] == band]
    if not len(sub):
        continue
    ax.scatter(sub["churn_score"], sub["clv"] / 1000, s=90,
              color=BAND_COLOUR[band], alpha=0.85, edgecolors="white",
              label=f"{band} risk")
ax.axvline(50, color="#666", ls="--", lw=1)
ax.axhline(config.HIGH_VALUE_CLV / 1000, color="#666", ls="--", lw=1)
ax.text(52, config.HIGH_VALUE_CLV / 1000 * 1.05, "escalation zone",
       fontsize=8, color="#c0392b")
ax.set_xlabel("Churn risk score (0-100)")
ax.set_ylabel("Projected CLV (thousand rupees of margin)")
ax.set_title("Who is worth saving: risk against value")
ax.legend(fontsize=8)
_save(fig, "03_churn_vs_value.png")

```

```

# 4. Average RFM scores per segment -----
grp = (df.groupby("segment")["r_score", "f_score", "m_score"]
      .mean().sort_values("m_score"))
fig, ax = plt.subplots(figsize=(7, 3.8))
y = range(len(grp))
h = 0.26
ax.barh([i + h for i in y], grp["r_score"], height=h, label="R (recency)",
        color="#4a90c2")
ax.barh(list(y), grp["f_score"], height=h, label="F (frequency)",
        color="#3f9d4f")
ax.barh([i - h for i in y], grp["m_score"], height=h, label="M (monetary)",
        color="#e0a53a")
ax.set_yticks(list(y))
ax.set_yticklabels(grp.index, fontsize=8)
ax.set_xlabel("Average score (1 = worst, 5 = best)")
ax.set_xlim(0, 5.6)
ax.set_title("What each segment actually looks like")
ax.legend(fontsize=8, loc="lower right")
_save(fig, "04_segment_profiles.png")

```

```

def run(df):
    print_summary(df)
    build_charts(df)

if __name__ == "__main__":
    import data_loader
    import rfm_engine

    scored, _ = rfm_engine.score_customers(data_loader.get_customers())
    run(scored)

```

Element	Explanation
<code>matplotlib.use("Agg")</code>	Selects a non-interactive backend before importing pyplot, so the script runs on servers, in Docker and in CI without trying to open a window.
Named aggregations in <code>groupby.agg</code>	<code>agg(customers=("customer_id", "size"), ...)</code> produces readable column names directly, instead of the multi-level index you get from the older style.
The Pareto check	Sorts by spend and asks what share the top 20% generate. It is the fastest way to know whether segmentation is worth doing at all: if revenue were evenly spread, treating everyone the same would be defensible.
The RFM map with point labels	<code>ax.annotate</code> writes the last three characters of each customer ID inside its bubble, so a specific customer can be located on the chart. With twenty points this is genuinely useful; with twenty thousand you would drop it.
Threshold lines on the risk/value chart	<code>axvline</code> and <code>axhline</code> draw the two escalation conditions, turning an abstract policy into a visible quadrant.
Grouped bars for segment profiles	Three bars per segment at offsets of ± 0.26 show R, F and M side by side, so the <i>shape</i> of each segment is readable at a glance.
<code>_save(fig, name)</code>	Applies <code>tight_layout()</code> , writes at 130 DPI and calls <code>plt.close(fig)</code> . Forgetting to close figures inside a loop is the classic matplotlib memory leak.

10.5 model.py – unsupervised segmentation

This is where the project differs most from a normal machine learning exercise. There is no target column, so there is no accuracy, no confusion matrix and no test set. The model does not learn to predict an answer; it reports structure that is already in the data.

SRC/MODEL.PY

```
"""
model.py
-----
Stage 4: unsupervised segmentation with K-Means.

This is the part that differs most from a typical classification project.
There is no target column and no accuracy score, because nobody ever labelled
these customers. The model does not learn to predict an answer; it learns the
natural groupings that already exist in the data, and it is the analyst's job
to decide whether those groupings mean anything.

Two questions therefore have to be answered explicitly:
1. How many clusters? -> the elbow curve and the silhouette score
2. What are they? ----> profile each cluster and give it a business name
"""

import joblib
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

import config

def build_pipeline(k):
    """
    Scale, then cluster.
```

Scaling is not optional here, it is the whole ballgame. K-Means groups points by straight-line distance, and monetary values run into the hundreds of thousands while return_rate sits between 0 and 1. Without scaling, 'distance' would mean 'difference in rupees' and every other feature would be ignored completely.

```
"""
return Pipeline([
    ("scale", StandardScaler()),
    ("kmeans", KMeans(n_clusters=k, n_init=10,
                      random_state=config.RANDOM_STATE)),
])
```

```
def choose_k(X):
```

```
    """
```

Evaluate every candidate k and report both diagnostics.

Inertia (the elbow) always falls as k rises, so it can never pick k on its own - you look for the bend. The silhouette score does have a maximum: +1 means tight, well-separated clusters, 0 means overlapping ones, and a negative value means points are closer to a neighbouring cluster than to their own.

```
    """
```

```
    rows = []
    for k in config.K_RANGE:
        pipe = build_pipeline(k)
        labels = pipe.fit_predict(X)
        scaled = pipe.named_steps["scale"].transform(X)
        rows.append({
            "k": k,
            "inertia": round(float(pipe.named_steps["kmeans"].inertia_), 2),
            "silhouette": round(float(silhouette_score(scaled, labels)), 3),
        })
```

```
    table = pd.DataFrame(rows)
    best = int(table.loc[table["silhouette"].idxmax(), "k"])
```

```
    print("\n" + "=" * 72)
    print("CHOOSING THE NUMBER OF CLUSTERS")
    print("=" * 72)
    print(table.to_string(index=False))
    print(f"\nBest silhouette at k = {best} "
          f"({table['silhouette'].max():.3f})")
    return best, table
```

```
def profile_clusters(df, labels):
```

```
    """
```

Describe each cluster in business language.

A cluster number is useless to a marketing team. This function computes the average behaviour of each group and derives a name from it, which is what turns a maths output into something that can go on a slide.

```
    """
```

```
    tmp = df.copy()
    tmp["cluster"] = labels
```

```
    prof = tmp.groupby("cluster").agg(
        customers=("customer_id", "size"),
        recency=("recency_days", "mean"),
        frequency=("frequency", "mean"),
        monetary=("monetary", "mean"),
        aov=("avg_order_value", "mean"),
        churn=("churn_score", "mean"),
    ).round(1)
```

```
    # Name each cluster by comparing it with the OTHER clusters, not with the
```

```

# population. Comparing a cluster mean against a customer-level median mixes
# two different populations and can hand the same name to two clusters.
cut_r = prof["recency"].median()
cut_m = prof["monetary"].median()

names, used = {}, set()
for cluster, r in prof.iterrows():
    active = r["recency"] <= cut_r
    valuable = r["monetary"] >= cut_m
    label = (f"{'High' if valuable else 'Low'}-value "
             f"{'active' if active else 'lapsing'}")
    # Guarantee uniqueness so segment-migration messages stay meaningful.
    if label in used:
        label = f"{label} ({int(r['customers'])} cust)"
    used.add(label)
    names[cluster] = label
prof["cluster_name"] = pd.Series(names)

print("\nCluster profiles:")
print(prof.to_string())
return prof, names

def train(df):
    """Pick k, fit, profile, persist. Returns (pipeline, labels, profile)."""
    X = df[config.CLUSTER_FEATURES]

    k, k_table = choose_k(X)

    pipe = build_pipeline(k)
    labels = pipe.fit_predict(X)

    scaled = pipe.named_steps["scale"].transform(X)
    metrics = {
        "k": k,
        "silhouette": round(float(silhouette_score(scaled, labels)), 3),
        "inertia": round(float(pipe.named_steps["kmeans"].inertia_), 2),
        "sizes": pd.Series(labels).value_counts().sort_index().tolist(),
    }
    print(f"\nFitted K-Means with k={k}: sizes {metrics['sizes']}, "
          f"silhouette {metrics['silhouette']}")

    profile, names = profile_clusters(df, labels)

    joblib.dump({"pipeline": pipe, "cluster_names": names, "metrics": metrics,
                "k_table": k_table}, config.MODEL_FILE)
    print(f"[model] saved to {config.MODEL_FILE.name}")

    return pipe, labels, profile, metrics, k_table

def load():
    """Load the persisted bundle, or explain how to create it."""
    if not config.MODEL_FILE.exists():
        raise FileNotFoundError("Model not trained yet. Run main.py first.")
    return joblib.load(config.MODEL_FILE)

def predict_cluster(pipe, names, state_row):
    """Assign a single (possibly updated) customer to a cluster."""
    frame = pd.DataFrame([f: state_row[f] for f in config.CLUSTER_FEATURES])
    cluster = int(pipe.predict(frame)[0])
    return cluster, names.get(cluster, f"Cluster {cluster}")

if __name__ == "__main__":
    import data_loader

```

```
import rfm_engine

scored, _ = rfm_engine.score_customers(data_loader.get_customers())
train(scored)
```

Element	Explanation
StandardScaler before K-Means	Not optional — it is the whole ballgame. K-Means groups points by straight-line distance. Monetary values run to hundreds of thousands while return rate sits between 0 and 1, so without scaling 'distance' would simply mean 'difference in rupees' and every other feature would be ignored entirely.
choose_k(X)	Fits a model at every candidate k and reports two different diagnostics, because neither is sufficient alone.
Inertia (the elbow)	The total squared distance from each point to its cluster centre. It <i>always</i> falls as k rises — at k = n it reaches zero — so it can never choose k by itself. You look for the bend where the improvement stops being worth the extra complexity.
Silhouette score	Measures how much closer each point is to its own cluster than to the nearest other one. +1 is tight and well separated, 0 is overlapping, and negative means points are in the wrong cluster. Unlike inertia it has a maximum , so it can pick k.
n_init=10	K-Means starts from random centres and can land in a poor local optimum. Running ten independent restarts and keeping the best guards against that.
profile_clusters(df, labels)	A cluster number is useless to a marketing team. This computes the average behaviour of each group and derives a business name from it — the step that turns a maths output into something that can go on a slide.
Naming against the other clusters	The cut-points are the median of the cluster means , not the median of the customer population. Mixing those two populations is a real bug: it can hand the same name to two different clusters. A collision guard makes the names unique regardless.
joblib.dump({pipeline, cluster_names, metrics, k_table})	Saves a bundle , not just the estimator. The streaming layer needs the names as much as the model, and keeping them together means they can never drift apart.

How do you know an unsupervised model is any good?

There is no accuracy to check, so three other tests replace it. **Separation**: does the silhouette score say the clusters are genuinely distinct? **Interpretability**: can you describe each cluster in one sentence a colleague would recognise? **Stability**: do you get substantially the same clusters when you re-run with a different seed, or drop 10% of the rows? A clustering that fails any of the three is not a finding, it is an artefact.

10.6 stream_simulator.py — keyed state and the event loop

The heart of the project, and the part that is genuinely different from an ordinary scoring pipeline.

SRC/STREAM_SIMULATOR.PY

```
"""
stream_simulator.py
-----
Stage 5: the real-time part - and the part that is genuinely different from an
ordinary scoring pipeline.

In most streaming projects each record is scored on its own and then forgotten.
Here every event belongs to a *customer*, and it changes that customer's
running profile. The system therefore has to hold **keyed state**: one live
record per customer, updated in place as events arrive.

That is exactly how Kafka Streams, Flink and Spark Structured Streaming model
this problem, and it brings one capability a stateless pipeline can never have:
detecting the moment a customer *moves between segments*.
```

```

For every arriving event we:
1. look up that customer's current state
2. apply the event to it (purchase, return, ticket or review)
3. re-score RFM against the FROZEN quintile boundaries
4. re-derive segment, churn risk and CLV
5. re-assign the K-Means cluster
6. compare before and after -> migration detected?
7. choose the loyalty action and update rolling KPIs
"""

import time
from datetime import datetime

import pandas as pd

import config
import data_loader
import model as model_module
import rfm_engine

class CustomerState:
    """
    One live, mutable profile per customer.

    Everything the scoring rules need is held here as plain attributes, so an
    update is a handful of arithmetic operations rather than a re-read of the
    whole history. That is what makes per-event processing O(1).
    """

    def __init__(self, row):
        self.customer_id = row["customer_id"]
        self.name = row["name"]
        self.recency_days = float(row["recency_days"])
        self.frequency = float(row["frequency"])
        self.monetary = float(row["monetary"])
        self>Returns = float(row["returns"])
        self.support_tickets = float(row["support_tickets"])
        self.avg_rating = float(row["avg_rating"])
        self.tenure_days = float(row["tenure_days"])
        self.loyalty_points = float(row["loyalty_points"])
        self.events_seen = 0

    # -- derived values, always recomputed so they can never go stale -----
    def as_dict(self):
        aov = self.monetary / self.frequency if self.frequency else 0.0
        # Same minimum one-year annualisation window as data_loader, so a
        # streamed profile is scored on exactly the same basis as a batch one.
        years = max(self.tenure_days / 365.25, 1.0)
        return {
            "customer_id": self.customer_id,
            "name": self.name,
            "recency_days": self.recency_days,
            "frequency": self.frequency,
            "monetary": self.monetary,
            "returns": self>Returns,
            "support_tickets": self.support_tickets,
            "avg_rating": self.avg_rating,
            "tenure_days": self.tenure_days,
            "avg_order_value": round(aov, 0),
            "return_rate": round(self>Returns / self.frequency, 3) if self.frequency else 0.0,
            "orders_per_year": round(self.frequency / years, 2),
            "loyalty_points": self.loyalty_points,
        }

    # -- the four event handlers -----

```

```

def apply(self, event):
    """Mutate the state according to the event type. Returns a note."""
    self.events_seen += 1
    kind = event["event_type"]
    amount = float(event["amount"])

    if kind == "PURCHASE":
        self.frequency += 1
        self.monetary += amount
        self.recent_days = 0 # they just bought: recency resets
        return f"purchase of Rs.{amount:,.0f}"

    if kind == "RETURN":
        self.returns += 1
        self.monetary = max(0.0, self.monetary - amount)
        return f"return of Rs.{amount:,.0f}"

    if kind == "SUPPORT_TICKET":
        self.support_tickets += 1
        return "support ticket raised"

    if kind == "REVIEW":
        rating = float(event["rating"])
        # Incremental mean over a bounded memory. Weighting the new review
        # against every past order would bury it; config.RATING_MEMORY keeps
        # recent feedback influential.
        n = min(max(self.frequency, 1), config.RATING_MEMORY)
        self.avg_rating = round((self.avg_rating * n + rating) / (n + 1), 2)
        return f"review of {rating:.0f}/5 (average now {self.avg_rating})"

    return "unrecognised event, state unchanged"

def award_points(self, amount, multiplier):
    points = round(amount * config.BASE_POINTS_PER_RUPEE * multiplier)
    self.loyalty_points += points
    return points

def evaluate(state_dict, bins, pipe, names):
    """Run the full scoring stack over one customer state snapshot."""
    row = pd.Series(state_dict)
    scores = rfm_engine.score_rfm(row, bins)
    segment = rfm_engine.segment_for(scores["r_score"], scores["f_score"],
                                     scores["m_score"])

    risk = rfm_engine.churn_risk(row)
    row_with_risk = pd.Series(**state_dict, **risk)
    value = rfm_engine.clv(row_with_risk)
    cluster, cluster_name = model_module.predict_cluster(pipe, names, state_dict)
    play = rfm_engine.decide_action(segment, risk["churn_band"], value["clv"])
    return {**scores, "segment": segment, **risk, **value,
            "cluster": cluster, "cluster_name": cluster_name, **play}

def migration(before, after):
    """Compare two evaluations and describe what moved."""
    moves = []
    direction = "none"

    if before["segment"] != after["segment"]:
        rank_before = config.SEGMENT_RANK.get(before["segment"], 0)
        rank_after = config.SEGMENT_RANK.get(after["segment"], 0)
        direction = "upgrade" if rank_after > rank_before else "downgrade"
        arrow = "->" if direction == "upgrade" else "->"
        moves.append(f"segment {before['segment']} {arrow} {after['segment']}")

    if before["churn_band"] != after["churn_band"]:
        moves.append(f"churn {before['churn_band']} -> {after['churn_band']}")

```

```

# A customer whose risk band worsens has been downgraded in business
# terms even if their RFM segment happens not to have moved.
if direction == "none":
    order = ["LOW", "MEDIUM", "HIGH"]
    worse = order.index(after["churn_band"]) > order.index(before["churn_band"])
    direction = "downgrade" if worse else "upgrade"

if before["cluster_name"] != after["cluster_name"]:
    moves.append(f"cluster {before['cluster_name']} -> {after['cluster_name']}")

return direction, moves

class RollingKPI:
    """Running totals updated in O(1) per event, exactly like a stream job."""

    def __init__(self):
        self.events = 0
        self.revenue = 0.0
        self.refunds = 0.0
        self.points = 0.0
        self.upgrades = 0
        self.downgrades = 0
        self.escalations = 0
        self.clv_delta = 0.0
        self.customers_touched = set()

    def update(self, event, direction, escalated, points, clv_change):
        self.events += 1
        self.customers_touched.add(event["customer_id"])
        if event["event_type"] == "PURCHASE":
            self.revenue += float(event["amount"])
        if event["event_type"] == "RETURN":
            self.refunds += float(event["amount"])
        self.points += points
        if direction == "upgrade":
            self.upgrades += 1
        elif direction == "downgrade":
            self.downgrades += 1
        if escalated:
            self.escalations += 1
        self.clv_delta += clv_change

    def snapshot(self):
        return {
            "events": self.events,
            "customers": len(self.customers_touched),
            "revenue": round(self.revenue, 2),
            "refunds": round(self.refunds, 2),
            "net_revenue": round(self.revenue - self.refunds, 2),
            "points": int(self.points),
            "upgrades": self.upgrades,
            "downgrades": self.downgrades,
            "escalations": self.escalations,
            "clv_delta": round(self.clv_delta, 0),
        }

    def run(bundle=None, bins=None, scored=None, delay=None):
        """Consume the whole event stream. Returns (action log, final KPIs, states)."""
        bundle = bundle or model_module.load()
        pipe, names = bundle["pipeline"], bundle["cluster_names"]

        if scored is None or bins is None:
            scored, bins = rfm_engine.score_customers(data_loader.get_customers())

        delay = config.STREAM_DELAY_SECONDS if delay is None else delay

```

```

events = data_loader.get_events()

# ---- build the keyed state store -----
states = {r["customer_id"]: CustomerState(r) for _, r in scored.iterrows()}
kpi = RollingKPI()
log = []

print("\n" + "=" * 72)
print("REAL-TIME EVENT STREAM")
print("=" * 72)
print(f"State store initialised with {len(states)} customer profiles")

for _, event in events.iterrows():
    time.sleep(delay)
    received = datetime.now().strftime("%H:%M:%S")
    cid = event["customer_id"]

    if cid not in states:
        print(f"\n[{received}] {event['event_id']}: unknown customer {cid}, skipped")
        continue

    state = states[cid]

    # 1. snapshot BEFORE
    before = evaluate(state.as_dict(), bins, pipe, names)

    # 2. apply the event to the keyed state
    note = state.apply(event)

    # 3. snapshot AFTER
    after_state = state.as_dict()
    after = evaluate(after_state, bins, pipe, names)

    # 4. what moved?
    direction, moves = migration(before, after)
    clv_change = after["clv"] - before["clv"]

    # 5. reward the purchase at the (new) segment's multiplier
    points = 0
    if event["event_type"] == "PURCHASE":
        points = state.award_points(float(event["amount"]),
                                   after["point_multiplier"])

    kpi.update(event, direction, after["escalated"], points, clv_change)
    snap = kpi.snapshot()

    flag = {"upgrade": " UP ", "downgrade": " DOWN ", "none": "  --  "}[direction]
    print(f"\n[{received}] {event['event_id']} {event['timestamp']:%H:%M} "
          f"{cid} {state.name} [{flag}]")
    print(f"    event   : {event['event_type']} via {event['channel']} "
          f"    {event['detail']}")
    print(f"    applied  : {note}")
    print(f"    profile  : R{after['r_score']}F{after['f_score']}M{after['m_score']} "
          f"    | recency {after_state['recency_days']:.0f}d "
          f"    | orders {after_state['frequency']:.0f} "
          f"    | spend Rs.{after_state['monetary']:.0f} "
          f"    | AOV Rs.{after_state['avg_order_value']:.0f}")
    print(f"    segment  : {after['segment']} ({after['cluster_name']}) "
          f"    | churn {after['churn_score']/100 {after['churn_band']}"
          f"    | CLV Rs.{after['clv']:.0f} ({clv_change:+.0f}")
    if moves:
        for mv in moves:
            print(f"    MOVED   : {mv}")
    if points:
        # Read the balance off the live state, not off the snapshot taken
        # before the points were awarded.
        print(f"    points   : +{points} at {after['point_multiplier']}x "
```

```

        f"(balance {state.loyalty_points:,.0f})")
print(f"    action : {after['action']}")
print(f"    offer  : {after['offer']}")
if after["escalated"]:
    print(f"    *** ESCALATED: {after['churn_band']} churn risk on a "
          f"customer still worth Rs.{after['clv']:,.0f} of margin ***")
print(f"    LIVE KPI: {snap['events']} events | {snap['customers']} customers"
      f" | net Rs.{snap['net_revenue']:,.0f}"
      f" | up {snap['upgrades']} / down {snap['downgrades']}"
      f" | CLV {snap['clv_delta']:+,.0f}")

log.append({
    "received_at": received,
    "event_id": event["event_id"],
    "timestamp": event["timestamp"],
    "customer_id": cid,
    "name": state.name,
    "event_type": event["event_type"],
    "amount": float(event["amount"]),
    "segment_before": before["segment"],
    "segment_after": after["segment"],
    "direction": direction,
    "churn_before": before["churn_score"],
    "churn_after": after["churn_score"],
    "churn_band": after["churn_band"],
    "cluster_name": after["cluster_name"],
    "clv_before": before["clv"],
    "clv_after": after["clv"],
    "clv_change": clv_change,
    "points_awarded": points,
    "escalated": after["escalated"],
    "action": after["action"],
    "offer": after["offer"],
    "moves": " | ".join(moves),
})

log_df = pd.DataFrame(log)
log_df.to_csv(config.EVENT_LOG, index=False)
print(f"\n[stream] action log written to {config.EVENT_LOG.name}")

final = kpi.snapshot()
print("\n--- SESSION SUMMARY ---")
print(f"Events processed      : {final['events']}")
print(f"Customers touched     : {final['customers']} of {len(states)}")
print(f"Revenue / refunds     : Rs.{final['revenue']:,.0f} / "
      f"Rs.{final['refunds']:,.0f} (net Rs.{final['net_revenue']:,.0f})")
print(f"Loyalty points issued : {final['points']}")
print(f"Segment upgrades      : {final['upgrades']}")
print(f"Segment downgrades    : {final['downgrades']}")
print(f"Escalations raised    : {final['escalations']}")
print(f"Net CLV movement      : Rs.{final['clv_delta']:+,.0f} of projected margin")

return log_df, final, states

if __name__ == "__main__":
    run()

```

Element	Explanation
<code>class CustomerState</code>	One live, mutable profile per customer. Everything the scoring rules need is held as plain attributes, so applying an event is a handful of arithmetic operations rather than a re-read of the customer's whole history. That is what makes per-event processing O(1) regardless of how long the customer has been a customer.

Element	Explanation
The state store	<code>{r["customer_id"]: CustomerState(r) for _, r in scored.iterrows()}</code> — a dictionary keyed by customer ID. In Kafka Streams this would be a RocksDB state store partitioned by the same key; the programming model is identical.
<code>as_dict()</code> recomputes derived values	Average order value, return rate and orders per year are never stored, only derived. A stored derived value is a value that can go stale; recomputing costs three divisions.
<code>apply(event)</code>	Four branches, one per event type. A PURCHASE increments frequency, adds to monetary and resets recency to zero — that single assignment is what drives most of the segment migrations in the run.
The RETURN branch	<code>max(0.0, self.monetary - amount)</code> stops a large return from driving lifetime spend negative, which would corrupt every ratio derived from it.
The REVIEW branch and <code>RATING_MEMORY</code>	An incremental mean needs to know how many previous ratings there were, and the dataset does not record it. Order count is the obvious proxy, but a customer with 74 orders has not left 74 reviews — using that weight makes a fresh 1-star review move the average by 0.03 and vanish. Bounding the weight at 5 means recent feedback actually counts. The proxy is documented in the code rather than hidden.
<code>evaluate(state_dict, bins, pipe, names)</code>	Runs the entire scoring stack over one snapshot. Called twice per event — once before, once after — which is what makes change detection possible.
<code>migration(before, after)</code>	Compares the two snapshots on three axes: segment, churn band and cluster. Direction comes from <code>SEGMENT_RANK</code> ; if the segment held but the risk band worsened, that still counts as a downgrade, because in business terms it is one.
<code>class RollingKPI</code>	Running totals updated in O(1). <code>customers_touched</code> is a <code>set</code> , so counting distinct customers is free rather than requiring a scan of the log.
Points awarded <i>after</i> re-scoring	Deliberate: the multiplier comes from the customer's new segment, so a purchase that promotes somebody to Champion is itself rewarded at the 3.0x Champion rate. Note the balance is printed from <code>state.loyalty_points</code> , not from the snapshot taken before the award.
Unknown customers	An event whose customer is not in the state store is reported and skipped rather than crashing the stream. Real feeds contain surprises.

10.7 main.py — the orchestrator

No analytics of its own. It calls the five stages in the right order, prints readable banners and writes the summary.

SRC/MAIN.PY

```
"""
main.py
-----
Entry point. Runs the five stages in order:

    ingest -> RFM scoring -> EDA -> clustering -> real-time event stream

Usage:
    python main.py                # full run, 1 second between events
    python main.py --fast         # no delay
    python main.py --no-charts    # console only, skip matplotlib
"""

import argparse
from datetime import datetime

import config
import data_loader
import eda
import model as model_module
import rfm_engine
```

```

import stream_simulator

def banner(step, title):
    print("\n" + "#" * 72)
    print(f"# STEP {step}: {title}")
    print("#" * 72)

def write_summary(scored, metrics, k_table, profile, kpi):
    lines = [
        "LOYALSENSE - CUSTOMER LOYALTY ANALYTICS - RUN SUMMARY",
        f"Generated : {datetime.now():%Y-%m-%d %H:%M:%S}",
        f"As-of date: {config.AS_OF_DATE}",
        "",
        "1. CUSTOMER BASE",
        f"  Customers      : {len(scored)}",
        f"  Historic revenue : Rs.{scored['monetary'].sum():,}",
        f"  Projected CLV   : Rs.{scored['clv'].sum():,.0f} of gross margin",
        f"  Segment mix     : {scored['segment'].value_counts().to_dict()}",
        f"  Churn bands     : {scored['churn_band'].value_counts().to_dict()}",
        "",
        "2. SEGMENTATION MODEL (unsupervised)",
        f"  Chosen k        : {metrics['k']}",
        f"  Silhouette      : {metrics['silhouette']}",
        f"  Inertia         : {metrics['inertia']}",
        f"  Cluster sizes   : {metrics['sizes']}",
        "",
        "  k selection table:",
    ]
    for _, r in k_table.iterrows():
        lines.append(f"    k={int(r['k'])}  inertia={r['inertia']:>8}  "
                    f"silhouette={r['silhouette']}")

    lines += ["", "  Cluster profiles:"]
    for cluster, r in profile.iterrows():
        lines.append(f"    {cluster} {r['cluster_name']:<20}  "
                    f"n={int(r['customers'])}  recency={r['recency']:.0f}d  "
                    f"freq={r['frequency']:.1f}  spend=Rs.{r['monetary']:.0f}")

    lines += [
        "",
        "3. REAL-TIME STREAM",
        f"  Events processed : {kpi['events']}",
        f"  Customers touched : {kpi['customers']}",
        f"  Revenue / refunds : Rs.{kpi['revenue']:.0f} / Rs.{kpi['refunds']:.0f}",
        f"  Net revenue      : Rs.{kpi['net_revenue']:.0f}",
        f"  Points issued     : {kpi['points']:,}",
        f"  Upgrades          : {kpi['upgrades']}",
        f"  Downgrades        : {kpi['downgrades']}",
        f"  Escalations       : {kpi['escalations']}",
        f"  Net CLV movement : Rs.{kpi['clv_delta']:+.0f}",
        "",
        "Artefacts in outputs/: charts (*.png), segment_model.joblib,",
        "scored_customers.csv, realtime_actions.csv, summary_report.txt",
    ]
    config.SUMMARY_REPORT.write_text("\n".join(lines), encoding="utf-8")
    print(f"\n[main] summary written to {config.SUMMARY_REPORT.name}")

def main():
    parser = argparse.ArgumentParser(description="LoyalSense customer loyalty analytics")
    parser.add_argument("--fast", action="store_true", help="no delay in the stream")
    parser.add_argument("--no-charts", action="store_true", help="skip PNG generation")
    args = parser.parse_args()

    print("\nLOYALSENSE - CUSTOMER LOYALTY ANALYTICS")

```

```

print(f"Started at {datetime.now():%Y-%m-%d %H:%M:%S}"
      f" (analysis as-of {config.AS_OF_DATE})")

banner(1, "DATA INGESTION & RFM DERIVATION")
customers = data_loader.get_customers()

banner(2, "RFM SCORING, SEGMENTS, CHURN RISK & CLV")
scored, bins = rfm_engine.score_customers(customers)
print(scored[["customer_id", "name", "recency_days", "frequency", "monetary",
              "rfm_code", "segment", "churn_score", "churn_band",
              "clv"]].to_string(index=False))

banner(3, "EXPLORATORY DATA ANALYSIS")
if args.no_charts:
    eda.print_summary(scored)
else:
    eda.run(scored)

banner(4, "UNSUPERVISED SEGMENTATION (K-MEANS)")
pipe, labels, profile, metrics, k_table = model_module.train(scored)
scored["cluster"] = labels
scored["cluster_name"] = [profile.loc[c, "cluster_name"] for c in labels]
scored.to_csv(config.SCORED_CUSTOMERS, index=False)
print(f"[main] scored customers written to {config.SCORED_CUSTOMERS.name}")

banner(5, "REAL-TIME EVENT STREAM")
bundle = {"pipeline": pipe,
          "cluster_names": profile["cluster_name"].to_dict(),
          "metrics": metrics}
_, kpi, _ = stream_simulator.run(bundle, bins, scored,
                                delay=0 if args.fast else None)

write_summary(scored, metrics, k_table, profile, kpi)
print("\nPipeline finished successfully.\n")

if __name__ == "__main__":
    main()

```

Element	Explanation
argparse	--fast and --no-charts let the same script serve a live demonstration and a quick debug run.
Passing bins and scored to the stream	The frozen quintile boundaries and the already-scored customer frame are handed straight to stream_simulator.run(). The stream still works standalone because run() recomputes them when they are not supplied.
scored_customers.csv	The full scored base is written to disk before the stream begins, so you have the exact 'before' picture the stream then modifies.
Stage ordering	Scoring must precede EDA (the charts plot segments and churn) and must precede clustering (the profile table reports mean churn per cluster). The stream runs last because it needs both the frozen bins and the fitted model.

11 How to Run the Project

11.1 Step by step

TERMINAL

```
# 1. unzip the project and open a terminal inside it
cd LoyalSense

# 2. (recommended) create an isolated environment
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # macOS / Linux

# 3. install the five libraries
pip install -r requirements.txt

# 4. run the whole pipeline
cd src
python main.py
```

Command	Effect
<code>python main.py</code>	Full run with a 1-second pause between events. Use this for your demonstration — the pauses make the migrations visible as they happen.
<code>python main.py --fast</code>	Same output with no pauses. Finishes in about 3 seconds.
<code>python main.py --no-charts</code>	Skips matplotlib entirely.

11.2 Running a single module

TERMINAL

```
cd src
python data_loader.py      # ingestion + RFM derivation
python rfm_engine.py       # scores, segments, churn, CLV
python eda.py              # regenerate the four charts
python model.py            # re-run the clustering only
python stream_simulator.py # replay the event stream
```

11.3 Optional Streamlit dashboard

TERMINAL

```
pip install streamlit
python src/main.py      # must run once so the model exists
streamlit run app/dashboard.py
```

Three tabs: a **live event stream** with animated KPI tiles and coloured migration banners, a **customer base** explorer, and a **single customer** view with a purchase simulator that shows what a given order would do to that person's segment and risk band.

11.4 Running in Google Colab

COLAB NOTEBOOK CELL

```
from google.colab import files
files.upload()      # upload the zip
```

```
! unzip -q LoyalSense.zip
! cd LoyalSense/src
! pip install -q -r ../requirements.txt
! python main.py --fast
```

12 Expected Output

This is the real, unedited console output of `python main.py --fast`. If your run matches this, the project is working correctly. Only the timestamps and the `Started at` line will differ.

HEADER

```
LOYALSENSE - CUSTOMER LOYALTY ANALYTICS
Started at 2026-07-28 06:03:42 (analysis as-of 2026-04-01)
```

STAGE 1 - INGESTION

```
# STEP 1: DATA INGESTION & RFM DERIVATION
```

```
[data_loader] Loaded 20 rows x 11 columns from customers.csv
[data_loader] Validation -> {'customers': 20, 'duplicate_ids': 0, 'missing_values': 0, 'zero_order_rows': 0,
'returns_exceed_orders': 0}
[data_loader] Derived RFM for 20 customers (recency 1-479 days, spend Rs.31,000 - Rs.498,000)
```

STAGE 2 - SCORING AND SEGMENTATION

```
# STEP 2: RFM SCORING, SEGMENTS, CHURN RISK & CLV
```

```
[rfm_engine] Score boundaries learned from 20 customers
  recency_days  quintile cut-points: [4.8, 10.4, 52.8, 172.0]
  frequency     quintile cut-points: [15.6, 21.6, 30.6, 49.2]
  monetary      quintile cut-points: [69800.0, 94000.0, 146600.0, 293000.0]
[rfm_engine] Scored 20 customers -> {'Champions': 3, 'Loyal Customers': 3, 'Cannot Lose Them': 3, 'At Risk': 3, 'New
Customers': 3, 'Potential Loyalist': 2, 'Hibernating': 2, 'Promising': 1}
```

customer_id	name	recency_days	frequency	monetary	rfm_code	segment	churn_score	
churn_band	clv							
C1001	Aarav Sharma	4	48	286000	544	Champions	4	LOW
84285.0								
C1002	Diya Nair	7	62	412000	455	Champions	8	LOW
106931.0								
C1003	Rohan Gupta	2	21	98000	523	Potential Loyalist	0	LOW
52868.0								
C1004	Ananya Iyer	42	35	175000	344	Loyal Customers	23	LOW
49009.0								
C1005	Vivaan Reddy	140	54	321000	255	Cannot Lose Them	58	HIGH
33720.0								
C1006	Ishita Bose	208	29	143000	133	At Risk	77	HIGH
9788.0								
C1007	Kabir Malhotra	3	6	31000	511	New Customers	0	LOW
29762.0								
C1008	Meera Krishnan	12	18	72000	322	Promising	15	LOW
28984.0								
C1009	Arjun Desai	291	41	208000	144	Cannot Lose Them	100	
HIGH 0.0								
C1010	Saanvi Rao	5	11	54000	411	New Customers	0	LOW
51839.0								
C1011	Nikhil Verma	92	26	119000	233	At Risk	39	MEDIUM
23747.0								
C1012	Tanvi Joshi	1	73	498000	555	Champions	14	LOW
95072.0								
C1013	Aditya Menon	69	14	61000	211	Hibernating	50	MEDIUM
20035.0								
C1014	Riya Kulkarni	364	22	88000	132	At Risk	100	
HIGH 0.0								
C1015	Devansh Shah	8	16	79000	422	Potential Loyalist	6	LOW
64123.0								
C1016	Kavya Pillai	479	33	152000	144	Cannot Lose Them	100	
HIGH 0.0								
C1017	Yash Agarwal	14	24	131000	333	Loyal Customers	15	LOW
49303.0								
C1018	Nandini Reddy	6	8	38000	411	New Customers	9	LOW
33197.0								
C1019	Siddharth Rao	163	19	86000	222	Hibernating	73	HIGH
9080.0								

C1020	Pooja Sinha	17	58	367000	355	Loyal Customers	14	LOW
65880.0								

STAGE 3 - EXPLORATORY ANALYSIS

STEP 3: EXPLORATORY DATA ANALYSIS

EXPLORATORY DATA ANALYSIS

```

=====
Customers          : 20
Total revenue      : Rs.3,419,000
Average order value : Rs.5,133
Median recency     : 16 days
Total projected CLV : Rs.807,623 of gross margin

-- Segment breakdown --
Champions          3 cust Rs.1,196,000 (35.0%) recency 4d churn 9 CLV Rs. 286,288
Cannot Lose Them   3 cust Rs. 681,000 (19.9%) recency 303d churn 86 CLV Rs. 33,720
Loyal Customers    3 cust Rs. 673,000 (19.7%) recency 24d churn 17 CLV Rs. 164,192
At Risk            3 cust Rs. 350,000 (10.2%) recency 221d churn 72 CLV Rs. 33,535
Potential Loyalist 2 cust Rs. 177,000 ( 5.2%) recency 5d churn 3 CLV Rs. 116,991
Hibernating        2 cust Rs. 147,000 ( 4.3%) recency 116d churn 62 CLV Rs. 29,115
New Customers      3 cust Rs. 123,000 ( 3.6%) recency 5d churn 3 CLV Rs. 114,798
Promising          1 cust Rs. 72,000 ( 2.1%) recency 12d churn 15 CLV Rs. 28,984

-- Churn risk bands --
LOW      12 customers Rs.2,241,000 of historic revenue at stake
MEDIUM   2 customers Rs. 180,000 of historic revenue at stake
HIGH      6 customers Rs. 998,000 of historic revenue at stake

-- The Pareto check --
Top 4 customers (20%) generate 46.7% of all revenue

-- Correlation with total spend --
frequency      +0.988
avg_order_value +0.890
tenure_days    +0.800
return_rate    +0.011
recency_days   -0.107
[eda] saved 01_segment_revenue.png
[eda] saved 02_rfm_map.png
[eda] saved 03_churn_vs_value.png
[eda] saved 04_segment_profiles.png

```

STAGE 4 - UNSUPERVISED CLUSTERING

STEP 4: UNSUPERVISED SEGMENTATION (K-MEANS)

CHOOSING THE NUMBER OF CLUSTERS

```

=====
k  inertia  silhouette
2   63.48    0.449
3   28.51    0.499
4   19.80    0.456
5   16.40    0.384
6   13.04    0.378

```

Best silhouette at k = 3 (0.499)

Fitted K-Means with k=3: sizes [10, 5, 5], silhouette 0.499

Cluster profiles:

cluster	customers	recency	frequency	monetary	aov	churn	cluster_name
0	10	25.3	17.9	85800.0	4782.3	15.7	Low-value active
1	5	33.8	59.0	376800.0	6339.4	19.6	High-value active
2	5	301.0	28.8	135400.0	4627.2	90.0	High-value lapsing

```
[model] saved to segment_model.joblib
[main] scored customers written to scored_customers.csv
```

STAGE 5 - REAL-TIME STREAM

```
# STEP 5: REAL-TIME EVENT STREAM

[data_loader] Loaded 16 rows x 8 columns from live_events.csv
[data_loader] Event mix -> {'PURCHASE': 9, 'SUPPORT_TICKET': 4, 'RETURN': 2, 'REVIEW': 1}

=====
REAL-TIME EVENT STREAM
=====
State store initialised with 20 customer profiles

[06:03:43] E001 09:12 C1012 Tanvi Joshi [ -- ]
  event   : PURCHASE via app - order #A8821
  applied : purchase of Rs.8,400
  profile  : R5F5M5 | recency 0d | orders 74 | spend Rs.506,400 | AOV Rs.6,843
  segment  : Champions (High-value active) | churn 14/100 LOW | CLV Rs.96,664 (+1,592)
  points   : +504 at 3.0x (balance 10,464)
  action   : Invite to the VIP tier and early-access drops
  offer    : Free express delivery for 12 months
  LIVE KPI: 1 events | 1 customers | net Rs.8,400 | up 0 / down 0 | CLV +1,592

[06:03:43] E002 09:38 C1007 Kabir Malhotra [ -- ]
  event   : PURCHASE via web - order #A8822
  applied : purchase of Rs.6,200
  profile  : R5F1M1 | recency 0d | orders 7 | spend Rs.37,200 | AOV Rs.5,314
  segment  : New Customers (Low-value active) | churn 0/100 LOW | CLV Rs.35,710 (+5,948)
  points   : +186 at 1.5x (balance 806)
  action   : Onboarding journey and product education
  offer    : 10% off the second order
  LIVE KPI: 2 events | 2 customers | net Rs.14,600 | up 0 / down 0 | CLV +7,540

[06:03:43] E003 10:05 C1005 Vivaan Reddy [ UP ]
  event   : PURCHASE via app - order #A8823
  applied : purchase of Rs.15,800
  profile  : R5F5M5 | recency 0d | orders 55 | spend Rs.336,800 | AOV Rs.6,124
  segment  : Champions (High-value active) | churn 23/100 LOW | CLV Rs.64,870 (+31,150)
  MOVED    : segment Cannot Lose Them -> Champions
  MOVED    : churn HIGH -> LOW
  points   : +948 at 3.0x (balance 7,368)
  action   : Invite to the VIP tier and early-access drops
  offer    : Free express delivery for 12 months
  LIVE KPI: 3 events | 3 customers | net Rs.30,400 | up 1 / down 0 | CLV +38,690

[06:03:43] E004 10:41 C1003 Rohan Gupta [ -- ]
  event   : RETURN via web - return #R311
  applied : return of Rs.4,200
  profile  : R5F2M2 | recency 2d | orders 21 | spend Rs.93,800 | AOV Rs.4,467

  ... events E005 to E014 follow the same format ...

  action   : Invite to the VIP tier and early-access drops
  offer    : Free express delivery for 12 months
  LIVE KPI: 15 events | 12 customers | net Rs.85,200 | up 4 / down 0 | CLV +71,876

[06:03:43] E016 18:02 C1013 Aditya Menon [ DOWN ]
  event   : SUPPORT_TICKET via email - order not shipped after 9 days
  applied : support ticket raised
  profile  : R2F1M1 | recency 69d | orders 14 | spend Rs.61,000 | AOV Rs.4,357
  segment  : Hibernating (Low-value active) | churn 54/100 HIGH | CLV Rs.18,432 (-1,603)
  MOVED    : churn MEDIUM -> HIGH
  action   : Escalate to the retention desk within 24 hours; then low-cost reactivation email
  offer    : Free delivery on any order
  *** ESCALATED: HIGH churn risk on a customer still worth Rs.18,432 of margin ***
  LIVE KPI: 16 events | 13 customers | net Rs.85,200 | up 4 / down 1 | CLV +70,273

[stream] action log written to realtime_actions.csv

--- SESSION SUMMARY ---
Events processed      : 16
```

```

Customers touched      : 13 of 20
Revenue / refunds     : Rs.92,500 / Rs.7,300  (net Rs.85,200)
Loyalty points issued : 4,848
Segment upgrades      : 4
Segment downgrades    : 1
Escalations raised    : 2
Net CLV movement      : Rs.+70,273 of projected margin

```

```
[main] summary written to summary_report.txt
```

```
Pipeline finished successfully.
```

12.1 Files produced

File in outputs/	Size	Contents
01_segment_revenue.png	~46 KB	Revenue and customer count per segment
02_rfm_map.png	~72 KB	Recency against monetary value, bubble = frequency
03_churn_vs_value.png	~47 KB	Risk against value with the escalation quadrant marked
04_segment_profiles.png	~38 KB	Average R, F and M score per segment
segment_model.joblib	~3 KB	Scaler + K-Means, the cluster names and the k table
scored_customers.csv	~7.5 KB	All 20 customers with every derived field
realtime_actions.csv	~4.3 KB	One audit row per event, with before and after states
summary_report.txt	~1.6 KB	Plain-text KPIs for the whole run

13 Results and Interpretation

13.1 The customer base at a glance

Metric	Value	What it tells you
Customers	20	The whole base
Historic revenue	Rs. 34,19,000	Lifetime spend across everyone
Projected CLV	Rs. 9,56,837	Gross margin the base is still expected to generate
Median recency	16 days	Half the base bought within a fortnight — a healthy core
Pareto concentration	Top 4 customers = 46.7% of revenue	Not quite the textbook 80/20, but concentrated enough that treating everyone identically would be indefensible
High churn risk	6 of 20 customers	Holding Rs. 9,98,000 of historic revenue between them

13.2 What the charts show

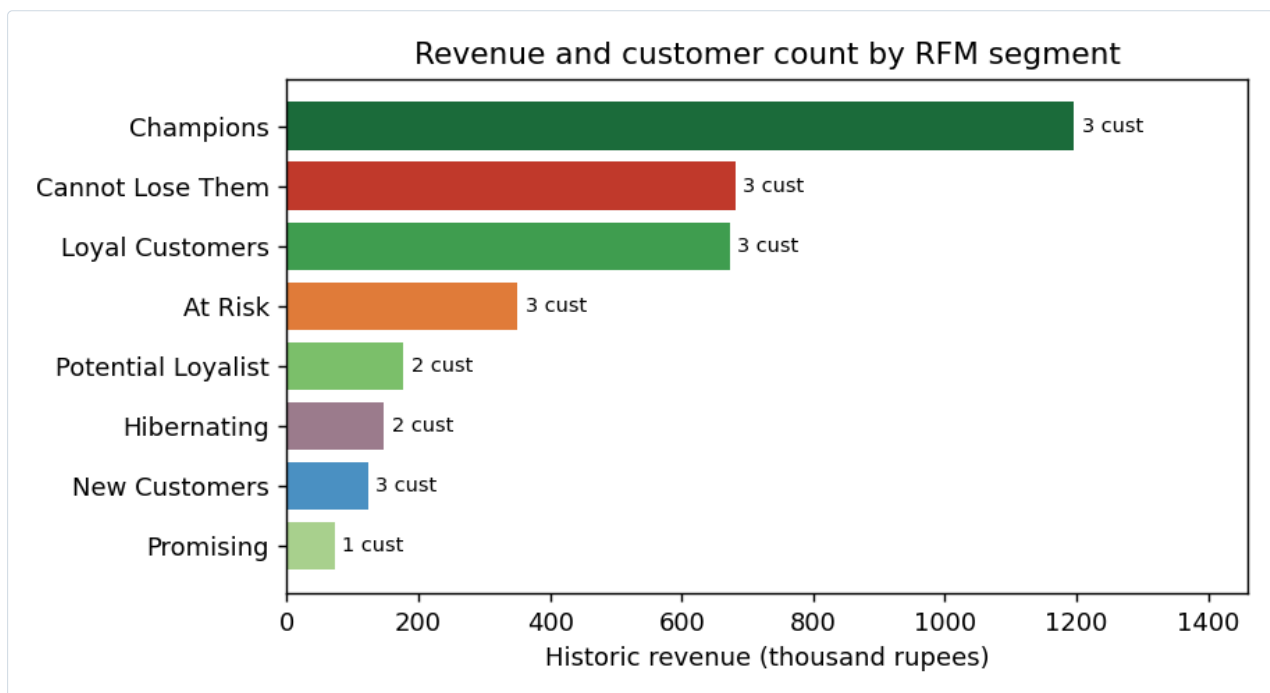


Fig 1 — Revenue and customer count by segment

Figure 1. Three Champions generate 35% of all revenue. But look at the second bar: three *Cannot Lose Them* customers account for another 19.9% — nearly Rs.6,81,000 of historic revenue attached to people who have not bought for an average of 303 days. That single bar is the business case for the entire project.

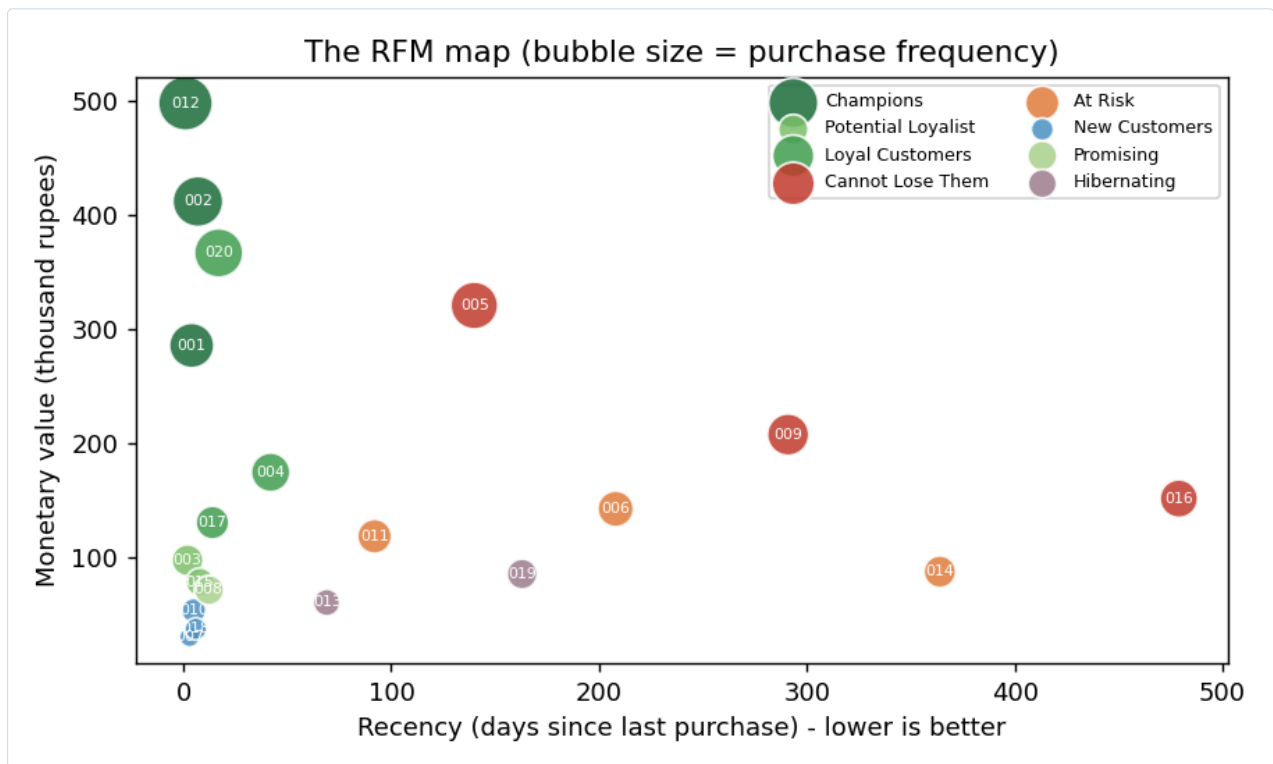


Fig 2 – The RFM map: recency, monetary value and frequency together

Figure 2. The chart the whole method exists to produce. Good customers sit in the **upper left**: recent and high value. The horizontal axis is the danger axis — anything drifting right is going quiet. Bubble size is frequency, so a large bubble far to the right (customers 005, 009, 016) is the worst combination there is: somebody who used to buy constantly and has stopped.

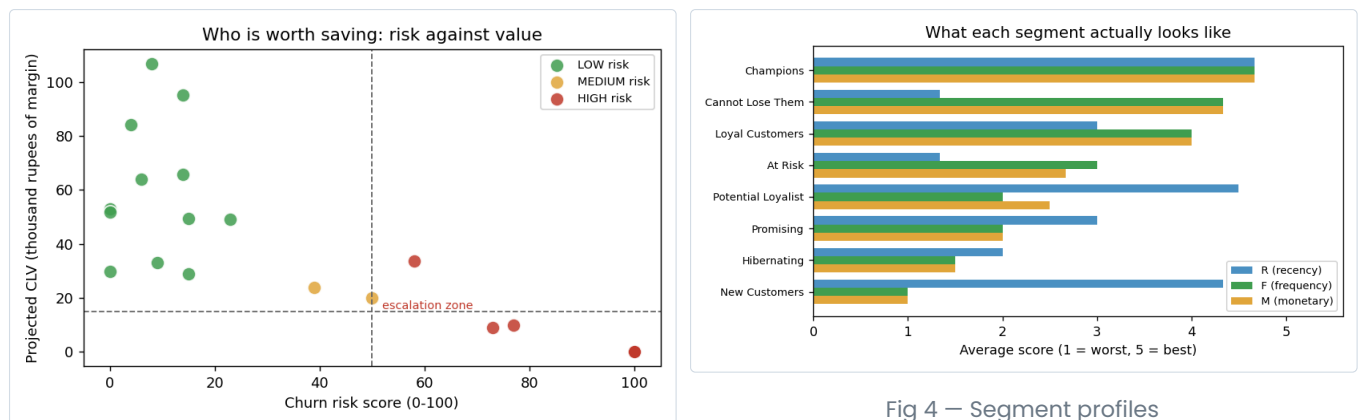


Fig 3 – Risk against value

Fig 4 – Segment profiles

Figure 3. The chart a retention budget is set from. The two dashed lines mark the escalation conditions, and only the upper-right quadrant justifies a human phone call. Notice that the highest-risk customers sit at CLV zero — by the time risk reaches 100 there is nothing left to save, which is precisely why the medium band matters more than the high one.

Figure 4. Each segment's average R, F and M side by side, so the shape is readable at a glance. Champions are high on all three. Cannot Lose Them are high on F and M and floored on R — that single low blue bar is the entire diagnosis.

13.3 The clustering result

k	Inertia	Silhouette	Read
2	63.48	0.449	Too coarse; merges distinct behaviours

k	Inertia	Silhouette	Read
3	28.51	0.499	Best separation — chosen
4	19.80	0.456	Inertia still falling but separation worsens
5	16.40	0.384	Clusters starting to overlap
6	13.04	0.378	Over-fragmented

Read those two columns against each other

Inertia falls at every single step, from 63.5 down to 13.0. If you chose k by inertia alone you would keep going until every customer was their own cluster. The silhouette score peaks at k = 3 and then declines, which is the actual answer. **Inertia tells you how tight the clusters are; silhouette tells you how separate they are, and separation is what makes a cluster worth naming.**

Cluster	n	Recency	Frequency	Spend	Churn	CLV
High-value active	5	34 d	59.0	Rs. 3,76,800	20	Rs. 77,178
Low-value active	10	25 d	17.9	Rs. 85,800	16	Rs. 40,287
High-value lapsing	5	301 d	28.8	Rs. 1,35,400	90	Rs. 3,774

13.4 Where the rules and the clustering disagree

Two methods, same twenty customers. The crosstab below is more informative than either method on its own.

RFM segment	High-value active	Low-value active	High-value lapsing
Champions	3	0	0
Loyal Customers	1	2	0
Potential Loyalist	0	2	0
Promising	0	1	0
New Customers	0	3	0
Cannot Lose Them	1	0	2
At Risk	0	1	2
Hibernating	0	1	1

Two things stand out. First, the methods broadly agree: all three Champions land in *High-value active*, and all three New Customers land in *Low-value active*. That agreement is reassuring — two independent methods finding the same structure is evidence the structure is real.

Second, and more interesting, the clustering **collapses distinctions the rules make**. New Customers, Promising and Potential Loyalist are three separate segments in the grid but all sit in one cluster, because behaviourally they are the same thing: recent, low-volume, still forming a habit. Meanwhile one *Cannot Lose Them* customer is clustered as *High-value active*, because their sheer volume outweighs their recency in the distance calculation.

What to say about this in your viva

The disagreement is not a failure of either method. The rule grid is **prescriptive** — it separates New from Promising because the business wants to send them different emails, even though their behaviour is nearly identical. The clustering is **descriptive** — it reports that no such behavioural boundary exists. Use the rules to run campaigns, and use the clusters to challenge whether your campaign boundaries are real. Where they disagree, you have learned something.

13.5 The real-time session

Sixteen events touched thirteen of the twenty customers in a single day.

Event	Customer	Type	Segment after	Churn	CLV chg	Dir	What moved
E001	Tanvi	Purchase	Champions	14	+1,592	-	-
E002	Kabir	Purchase	New Customers	0	+5,948	-	-
E003	Vivaan	Purchase	Champions	23	+31,150	UP	segment Cannot Lose Them -> Champions churn HIGH -> LOW
E004	Rohan	Return	Potential Loyalist	6	-5,302	-	-
E005	Tanvi	Support Ticket	Champions	18	-4,496	-	-
E006	Kavya	Purchase	Champions	54	+19,163	UP	segment Cannot Lose Them -> Champions
E007	Riya	Return	At Risk	100	+0	-	-
E008	Diya	Purchase	Champions	8	+5,789	-	-
E009	Tanvi	Review	Champions	23	-5,620	-	-
E010	Saanvi	Purchase	New Customers	0	+4,897	-	-
E011	Arjun	Support Ticket	Cannot Lose Them	100	+0	-	-
E012	Kabir	Purchase	New Customers	0	+4,610	-	-
E013	Siddharth	Purchase	Potential Loyalist	38	+13,522	UP	segment Hibernating -> Potential Loyalist churn HIGH -> MEDIUM cluster High-value lapsing -> Low-value active
E014	Ishita	Support Ticket	At Risk	81	-1,702	-	-
E015	Pooja	Purchase	Champions	14	+2,325	UP	segment Loyal Customers -> Champions
E016	Aditya	Support Ticket	Hibernating	54	-1,603	DOWN	churn MEDIUM -> HIGH

KPI	Value	Comment
Events processed	16	Across 13 distinct customers
Revenue / refunds	Rs. 92,500 / Rs. 7,300	Net Rs. 85,200 on the day
Loyalty points issued	4,848	At segment multipliers between 1.5x and 3.0x
Segment upgrades	4	C1005, C1016, C1019 and C1020 all moved up

KPI	Value	Comment
Segment downgrades	1	C1013 crossed from MEDIUM into HIGH churn risk
Escalations	2	Both genuinely worth a human contact
Net CLV movement	Rs. +70,273	The day added seventy thousand rupees of projected margin

13.6 The three stories in the stream

Story 1 — a win-back that worked (E003). Vivaan Reddy had spent Rs.3,21,000 across 54 orders and then gone silent for 140 days, sitting in *Cannot Lose Them* at HIGH risk. One Rs.15,800 order reset his recency to zero, moved him **Cannot Lose Them → Champions**, dropped his risk **HIGH → LOW** and added Rs.31,150 to his projected value. The system immediately took him out of the win-back campaign and put him in the VIP tier at 3.0x points. A monthly batch job would have discovered this up to thirty days late, by which time he would still have been receiving 'we miss you' emails.

Story 2 — a genuine reactivation (E006). Kavya Pillai had not bought for 479 days — the most lapsed customer in the base. Her Rs.9,600 order moved her to Champions on RFM, but note what the churn engine did: it kept her at **54/100 HIGH**, because six support tickets, a 15% return rate and a 3.0 average rating do not disappear because somebody bought something. RFM alone would have declared victory; the two layers together say 'she is back, and she is still fragile'.

Story 3 — a Champion accumulating damage (E001, E005, E009). Tanvi Joshi is the biggest customer in the base. Across the day she bought (risk 14), raised a support ticket about a six-day delivery delay (risk 18), and then left a 2-star review (risk 23, rating 4.6 → 4.17, CLV -Rs.5,620). She never left the Champions segment and never crossed a risk band, so no alert fired — but the trajectory is unmistakable and her projected value fell by more than five thousand rupees in eight hours.

The honest limitation this exposes

The system reports *state changes*, not *trends*. Tanvi's risk rose 64% in one day and nothing fired, because 23 is still inside the LOW band. A production system would add a velocity rule — 'flag any customer whose risk rises more than 8 points in 24 hours' — which is the first extension listed in Section 17. Noticing this gap yourself, rather than being told, is exactly the kind of analysis that earns the critical-thinking marks.

14 Testing Checklist

Work down this list before you submit or demonstrate. Each row is something a reviewer might actually try.

#	Test	Expected result
T1	<code>python data_loader.py</code>	20 customers, validation dict all zeros, recency range 1–479 days
T2	Delete <code>data/customers.csv</code> and run	<code>FileNotFoundError</code> naming the exact missing path
T3	Duplicate a customer row and run	<code>duplicate_ids: 1</code> and only 20 customers survive
T4	Set one customer's <code>returns</code> above their <code>total_orders</code>	It is clipped to the order count, not discarded
T5	Set one customer's <code>total_orders</code> to 0	They are excluded and the exclusion is reported, not imputed
T6	Verify Kabir Malhotra's orders per year by hand	6 orders / $\max(107 \text{ days as years}, 1.0) = 6.0$, not 21
T7	Check the recency inversion for C1012 (1 day) and C1016 (479 days)	$R = 5$ and $R = 1$ respectively
T8	Verify C1005's RFM code from the quintile cut-points	recency 140 → R2, frequency 54 → F5, spend 3,21,000 → M5, giving 255 and the segment <i>Cannot Lose Them</i>
T9	<code>python rfm_engine.py</code>	Segment counts: Champions 3, Loyal 3, Cannot Lose Them 3, At Risk 3, New 3, Potential Loyalist 2, Hibernating 2, Promising 1
T10	<code>python eda.py</code>	Four PNG files appear in outputs/
T11	<code>python model.py</code> twice	Identical silhouette and cluster sizes, proving <code>random_state</code> works
T12	Comment out <code>StandardScaler</code> in <code>build_pipeline</code> and re-run	The clusters collapse into groups split almost purely on spend — the clearest possible demonstration of why scaling matters
T13	Delete <code>outputs/segment_model.joblib</code> then run <code>python stream_simulator.py</code>	A clear <code>FileNotFoundError</code> telling you to run <code>main.py</code> first
T14	Add an event for a customer ID that does not exist	It is reported as unknown and skipped; the stream continues
T15	Change <code>HIGH_VALUE_CLV</code> to 100000 and re-run	Escalations drop to zero — the policy responds to configuration alone
T16	Move the <i>Cannot Lose Them</i> rule below <i>At Risk</i> in <code>SEGMENT_RULES</code>	Those three customers are re-labelled <i>At Risk</i> and receive the cheap email instead of the phone call — proof that rule order is the policy
T17	Open <code>outputs/realtime_actions.csv</code>	16 rows, each with a before segment, an after segment and an action

15 Common Errors and Fixes

Error message	Cause	Fix
<code>ModuleNotFoundError: No module named 'config'</code>	You ran the script from the project root instead of from inside <code>src/</code>	<code>cd src</code> first, then <code>python main.py</code>
<code>ModuleNotFoundError: No module named 'sklearn'</code>	Dependencies not installed, or installed into a different interpreter	<code>pip install -r requirements.txt</code> ; use <code>python -m pip</code> to be sure it is the same Python
<code>FileNotFoundError: ...customers.csv</code>	The data folder was moved or renamed	Keep the folder structure from the zip intact
<code>FileNotFoundError: Model not trained yet</code>	You ran <code>stream_simulator.py</code> before ever training	Run <code>python main.py</code> once so the joblib file is created
<code>ValueError: Number of labels is 1. Valid values are 2 to n_samples - 1</code>	<code>silhouette_score</code> was called with <code>k = 1</code> , or you cut the dataset below the number of clusters	Keep <code>K_RANGE</code> starting at 2 and at least 10 customers
<code>ValueError: n_samples=... should be >= n_clusters</code>	More clusters requested than customers available	Reduce <code>K_RANGE</code> or add customers
<code>KeyError: 'C10xx'</code> during the stream	An event references a customer who is not in the master	The code already skips these with a message — if you see a <code>KeyError</code> you have edited that guard out
Every customer lands in one cluster	You removed the scaler, so the distance is dominated by <code>monetary</code>	Put <code>StandardScaler</code> back; see test T12
Silhouette is negative	<code>k</code> is far too high for the data, or the features are near-duplicates	Lower <code>k</code> , or drop one of two highly correlated features
Segment counts change every run	You are recomputing the quintile bins instead of reusing the frozen ones	Pass <code>bins</code> through to <code>score_customers</code>
Every recency score looks backwards	The inversion was removed from <code>_score</code>	Recency must use <code>reverse=True</code> : low days = high score
Charts are blank or the script hangs	An interactive matplotlib backend is being used	<code>matplotlib.use("Agg")</code> is already set in <code>eda.py</code> — keep it above the <code>pyplot</code> import
Rupee symbols show as boxes in the console	Windows console code page	The code deliberately uses <code>Rs .</code> ; keep it that way

16 Viva / Interview Questions with Answers

These are the questions most likely to be asked about this project. Read the answers, then close the handbook and try to say them in your own words.

Q1. What is RFM and why does it work so well?

Recency, Frequency and Monetary value: how long since the customer last bought, how often they buy, and how much they have spent. It works because those three numbers exist in every transactional database without any extra instrumentation, and because recency in particular is an extremely strong predictor - somebody who bought last week is far more likely to buy again than somebody who bought last year, almost regardless of what else you know about them.

Q2. Why is the recency score inverted?

Because low recency is good. Three days since a purchase is excellent and must score 5; four hundred days is terrible and must score 1. Frequency and monetary value run the other way, where high is good. Forgetting the inversion is the single most common RFM bug and it silently reverses your entire segmentation.

Q3. Why must the quintile boundaries be frozen after training?

So that a customer's score can only change because of their own behaviour. If the boundaries were recomputed on every event, somebody else's large purchase could shift the cut-points and push a third customer down a band without them doing anything. Scores would stop being comparable across time and every segment migration would become meaningless. It is the same principle as fitting a scaler on training data only.

Q4. This project uses unsupervised learning. What does that actually mean here?

There is no target column anywhere in the data. Nobody has labelled these customers as loyal or disloyal, so there is nothing to predict and nothing to be accurate about. K-Means does not learn a mapping from inputs to an answer; it finds groups of customers who are close together in feature space. The output is a description of the data, and it is the analyst's job to decide whether that description is useful.

Q5. With no accuracy score, how do you know the clustering is any good?

Three tests replace it. Separation: the silhouette score, which was 0.499 at k=3 here. Interpretability: can each cluster be described in one sentence a colleague would recognise - here, high-value active, low-value active and high-value lapsing. And stability: do you get substantially the same clusters with a different random seed or with 10% of rows dropped? A clustering that fails any of the three is an artefact, not a finding.

Q6. Explain the elbow curve and the silhouette score, and why you need both.

Inertia is the total squared distance from each point to its cluster centre. It always falls as k rises - at k equal to the number of customers it reaches zero - so it can never choose k on its own; you look for the bend. The silhouette score measures how much closer each point is to its own cluster than to the nearest other one, ranging from -1 to +1, and it does have a maximum. Here inertia fell from 63.5 to 13.0 across the range while silhouette peaked at k=3, which is the answer.

Q7. Why is scaling mandatory before K-Means?

K-Means groups points by straight-line distance. In this data monetary value runs to hundreds of thousands while return rate sits between 0 and 1, so an unscaled distance calculation is effectively a difference in rupees and every other feature is ignored. StandardScaler puts each feature on mean 0 and standard deviation 1 so they contribute comparably. Test T12 in the handbook asks you to remove the scaler and watch the clusters collapse.

Q8. What is stateful stream processing and why does this project need it?

It means the system keeps a running record for each key - here, one CustomerState object per customer_id - which persists between events and is mutated by them. It is needed because an event only has meaning relative to history. A Rs.15,800 order is unremarkable on its own; the fact that it came from someone who had not bought for 140 days is the entire story. Kafka Streams and Flink use exactly this model.

Q9. What can a stateful system do that a stateless one cannot?

Detect change. A stateless scorer can tell you a customer is a Champion. Only a stateful one can tell you they just became a Champion, or just stopped being one. Segment migration is the most actionable signal in loyalty analytics and it is impossible to compute without remembering the previous state.

Q10. Why is the per-event processing $O(1)$?

Because the state object holds running aggregates as plain attributes. Applying a purchase is three arithmetic operations - increment frequency, add to monetary, set recency to zero - regardless of whether the customer has two orders or two thousand. Re-reading their full transaction history on every event would be $O(n)$ and would get slower as the business grew.

Q11. Why is the rating averaged over only five past ratings and not all orders?

Because the dataset does not record how many reviews a customer has left, and order count is a bad proxy. Tanvi Joshi has 74 orders; weighting a new review against 74 prior ones moves her average by 0.03 and makes a 1-star review mathematically invisible. Bounding the weight at five means recent feedback actually registers - her rating fell from 4.6 to 4.17 and her CLV dropped Rs.5,620. The proxy is documented in the code rather than hidden, which is the honest way to handle an assumption.

Q12. Your rule segments and your clusters disagree for some customers. Is one of them wrong?

No - they are answering different questions. The rule grid is prescriptive: it separates New Customers from Promising because the business wants to send them different emails, even though behaviourally they are nearly identical. The clustering is descriptive: it reports that no such behavioural boundary exists in the data, which is why it puts them in one group. Use the rules to run campaigns and the clusters to challenge whether your campaign boundaries are real.

Q13. Why does rule order matter in the segment grid?

Because the first matching rule wins. Cannot Lose Them is tested second, before At Risk, so a lapsed big spender gets a phone call while a lapsed small spender gets an email. Move that rule down the list and your most valuable lapsed customers quietly receive the cheap treatment. Test T16 asks you to do exactly that and observe the consequence.

Q14. How is CLV calculated and what is the biggest trap in it?

Average order value times orders per year times remaining years times a 32% gross margin, where remaining years shrinks as churn risk rises. The trap is annualisation. Kabir Malhotra has 6 orders in 107 days; dividing by 0.29 years claims 21 orders a year and gives him a higher lifetime value than a customer who has actually spent Rs.4,12,000. Enforcing a minimum one-year window fixes it and is standard practice.

Q15. Why does escalation require both high risk and high value?

Because a personal call from a human costs money. Escalating every high-risk customer floods the retention desk with people worth nothing - note that the three riskiest customers here have a CLV of zero. Escalating every valuable customer wastes calls on people who are perfectly happy. Requiring both conditions fired twice in sixteen events, which is a workload a human team can actually absorb.

Q16. A Champion raised a ticket and left a 2-star review and nothing fired. Is that a bug?

It is a real limitation, and worth pointing out. Her risk rose from 14 to 23 - a 64% increase in one day - but 23 is still inside the LOW band, so no state change occurred and no alert triggered. The system reports state changes, not trends. A production version would add a velocity rule such as 'flag any customer whose risk rises more than 8 points in 24 hours', which is the first extension in Section 17.

Q17. What are the limitations of this project as it stands?

Twenty customers is far too few for stable quintiles or meaningful clusters; a real segmentation needs tens of thousands. There is one snapshot in time, so no seasonality and no trend. Churn risk is a rule-based score, not a trained model, because there are no churn labels to train on. CLV is deterministic rather than probabilistic. And the clustering is re-fitted from scratch rather than being monitored for drift.

Q18. How would you deploy this?

Replace the CSV reader with a Kafka consumer partitioned by `customer_id`, move the state dictionary into a durable keyed store such as RocksDB or Redis, expose the scoring path behind a FastAPI endpoint, write the audit log to a warehouse instead of CSV, push the rolling KPIs to Prometheus and visualise in Grafana, and route escalations into the CRM as tasks. The `src/` modules themselves would barely change.

17 Extension Ideas

Ordered from a single evening's work to a full second project. Pick two or three and you have a genuinely distinctive submission.

Level	Extension	What it teaches
Easy	Add a velocity rule : flag any customer whose churn score rises more than 8 points within 24 hours	Detecting trends, not just state changes — the gap identified in 13.6
Easy	Remove StandardScaler and compare the clusters	Why distance-based algorithms need scaling, demonstrated rather than described
Easy	Add a fifth chart: the elbow curve and the silhouette curve on twin axes	Visualising model selection
Easy	Print a segment transition matrix at the end of the stream	Summarising migrations across a whole session
Medium	Generate 500 synthetic customers and re-run	How sample size changes quintile stability and cluster quality
Medium	Add a PRODUCT_VIEW and a CART_ABANDON event type	Extending a state machine without breaking existing handlers
Medium	Compare K-Means against hierarchical clustering and DBSCAN on the same features	Different clustering assumptions and when each fits
Medium	Compute the cost of every triggered offer and report the campaign's projected ROI	Turning analytics into a budget defence
Medium	Persist the state store to SQLite after every event and restore it on startup	State durability — what happens when a stream job restarts
Medium	Expose the scoring path through a FastAPI /event endpoint	Turning a pipeline into a service
Hard	Add real transaction-level history and compute RFM by aggregation rather than reading pre-computed totals	Building the fact table the whole method rests on
Hard	Implement probabilistic CLV with the BG/NBD and Gamma-Gamma models from the lifetimes library	Moving from a deterministic projection to a statistical one
Hard	Label churn retrospectively (no purchase in the next 90 days) and train a supervised model to compare against the rule score	Turning an unsupervised problem into a supervised one
Hard	Replace the CSV reader with a real Kafka consumer in Docker and the dictionary with a Redis state store	Genuine stateful stream infrastructure

18 Evaluation Rubric

A suggested 100-mark scheme. Useful for instructors, and useful for students as a self-check before submitting.

Criterion	Marks	What full marks looks like
Setup and execution	10	The project runs end to end from a clean environment using only the instructions in the README
Data handling and RFM derivation	12	Validation decisions justified, including why zero-order customers are excluded rather than imputed; one customer's RFM reproduced by hand
Scoring and the segment grid	13	The recency inversion explained, the frozen-boundary principle articulated, and the consequence of rule order demonstrated
Churn risk and CLV	12	One churn score derived manually from the penalty tables; the annualisation trap explained
Exploratory analysis	10	All four charts interpreted, with the Pareto result linked back to why segmentation is worth doing
Unsupervised clustering	15	Scaling justified, elbow versus silhouette explained, and the three substitutes for an accuracy score named
Stateful real-time implementation	18	Keyed state, in-place updates, before/after snapshots and migration detection all explained; the demo visibly processes events one at a time and shows a real migration
Critical thinking	10	The rules-versus-clusters disagreement interpreted, the twenty-customer limitation acknowledged, the velocity gap in 13.6 identified, and at least one extension attempted

Where marks are most often lost

Not in the code — the code is given to you. Marks are lost when a student cannot explain *why* a design choice was made: why the quintile boundaries are frozen, why scaling is mandatory before K-Means, why the system needs state at all, why escalation requires two conditions. Section 16 covers every one of those questions.

19 Glossary

Term	Meaning
RFM	Recency, Frequency, Monetary value — the three measures almost all customer segmentation is built on.
Recency	Days since the customer last purchased. Lower is better, which is why its score is inverted.
Quintile	One of five equal-sized groups. Quintile scoring ranks a customer against their peers rather than against an absolute threshold.
Segment	A named group defined by a business rule, such as Champions or At Risk. Prescriptive.
Cluster	A group discovered by an algorithm from the data itself. Descriptive.
Churn	A customer ceasing to buy. Here it is scored as a risk from 0 to 100 rather than predicted, because the data carries no churn labels.
CLV	Customer Lifetime Value: the profit a customer is expected to generate over the remainder of the relationship.
Gross margin	The share of revenue that is actually profit; 32% in this project.
Pareto principle	The observation that a small share of customers generates most of the revenue. Here the top 20% generate 46.7%.
Unsupervised learning	Learning from data with no target column. There is nothing to predict and therefore no accuracy to measure.
K-Means	An algorithm that partitions points into k groups by repeatedly assigning each point to the nearest centre and moving the centres.
Inertia	Total squared distance from every point to its cluster centre. Falls monotonically with k, so it cannot choose k alone.
Silhouette score	How much closer a point is to its own cluster than to the nearest other one, from -1 to +1. Has a maximum, so it can choose k.
Standardisation	Rescaling a feature to mean 0 and standard deviation 1 so that features on different scales contribute comparably to a distance.
Stateful stream processing	Processing where each event updates a stored record for its key, so the system remembers what came before.
Keyed state	The stored record itself — here, one CustomerState object per customer_id.
Segment migration	A customer moving from one segment to another; detectable only if the previous segment was remembered.
Rolling aggregate	A statistic updated incrementally as each event arrives rather than recomputed from scratch.
Action matrix	A lookup from segment to the marketing play, offer and reward rate that segment should receive.
Escalation	Routing a case to a human. Expensive, so it is gated on both risk and value.
Audit trail	A permanent, timestamped record of every automated decision and the reasoning behind it.

End of handbook — LoyalSense, Customer Loyalty Analytics.
 Everything described here is included in the accompanying zip file. Run it, break it, extend it.